Implementación y análisis del desempeño de un pipeline para el estudio de amplicones 16S en un clúster energéticamente eficiente basado en Raspberry Pi 5

Cristian Alberto Solano Torres 2211906 - Ingeniería de Sistemas

Zamir Francisco Granados Peñaloza 2211888 - Ingeniería de Sistemas

Trabajo de Grado para Optar el Título de Ingeniero de Sistemas

Director

Bernardo Andrés Benavides Arévalo. PhD - Director

Universidad Industrial de Santander

Facultad de Ingenierías Físico-Mecánicas

Escuela de Ingeniería de Sistemas e Informática

Bucaramanga

2026

Agradecimientos

Un sincero agradecimiento al director de este proyecto de grado, el profesor Bernardo Andrés Benavides Arévalo, por su acompañamiento durante el desarrollo de esta investigación. Su orientación técnica, sus observaciones y su disposición para discutir cada una de las decisiones que surgieron durante el proyecto fueron fundamentales para avanzar y superar los diferentes retos que encontramos en el camino. Agradecemos también la confianza que depositó en nosotros al permitirnos desarrollar un proyecto que nos llevó a explorar un campo interdisciplinario entre la ingeniería de sistemas y la bioinformática, y que representó una oportunidad para poner a prueba y fortalecer los conocimientos adquiridos durante nuestra formación.

A la Escuela de Ingeniería de Sistemas e Informática y a la Universidad Industrial de Santander, por acompañarnos durante los años en los que construimos las bases de nuestra formación como ingenieros. Agradecemos los conocimientos, las experiencias y los espacios que nos permitieron aprender no solo a desarrollar soluciones desde la ingeniería, sino también a cuestionarlas, mejorarlas y asumir con responsabilidad los problemas que se presentan en nuestra profesión. Este proyecto representa, en buena medida, el resultado de todo ese proceso de aprendizaje y de los años que compartimos como estudiantes.

Cristian Alberto Solano Torres

Agradezco especialmente a mis padres, Rosalba Torres Orejarena y Luis Alberto Solano Pinzón, por su amor, apoyo y confianza incondicional. Gracias por estar siempre a mi lado, por sostenerme en los momentos difíciles y, sobre todo, por no dejarme rendir cuando yo mismo llegué a pensar en hacerlo. Este logro también es de ustedes, porque detrás de cada paso que me trajo hasta aquí estuvieron su esfuerzo, sus consejos y la confianza que siempre depositaron en mí.A mis hermanas, Karen y Silvia, por estar pendientes de mí y acompañarme durante este proceso. Son ejemplos de superación y logro que me han ayudado a mantener

presente mi objetivo en todo momento. Y a mi sobrina, Shara, por ser esa pequeña fuente de amor que ha mantenido unida a toda la familia y por tenerme presente en todos los pasos que da.

A Zamir, mi compañero, por su esfuerzo y por dar ese impulso que muchas veces me hizo falta para seguir adelante. A Juan, mi mejor amigo, por ayudarme a mantener una actitud positiva y encontrar momentos de tranquilidad cuando todo parecía estar de cabeza, incluso después de aquellos momentos en los que estuvimos distanciados. Y a Daniel, por acompañarme cuando sentía que no podía salir de mis propios enredos y por ayudarme a encontrar claridad en esos momentos.

Finalmente, quiero dedicar un recuerdo especial a mis abuelos, Abigail Orejarena de Torres y Noé Solano Rueda, quienes partieron durante estos años de formación y no pudieron acompañarme hasta este momento. Este logro también lleva una parte de ellos y de los recuerdos que dejaron en mi vida.

Zamir Francisco Granados Peñaloza

Agradezco especialmente a mi familia. A mi madre, Patricia Peñaloza, quien me guio en cada paso y ha sido siempre mi consejera y mi refugio. A mis hermanos, Ronaldo, Heberth y Mabel, por acompañarme a lo largo de esta etapa con sus ocurrencias y con cada uno de sus favores. A Paula, mi compañera de vida, por comprenderme, apoyarme, escucharme y estar siempre para mí. A la memoria de Duke, mi amigo perruno, que me acompañó en los comienzos de este camino y me escuchaba en los momentos difíciles.

A mi compañero Cristian, por ayudarme cuando algo de la universidad no me quedaba claro y por ser parte de este cierre. A mi amigo Jaider, por su apoyo y por ser una persona leal. A Yeison, por ser un amigo siempre dispuesto a ayudar. Y a mi amigo Rosemberg, que estuvo conmigo desde los inicios de mi carrera.

A mi padre, Francisco Granados, mi mayor ejemplo de vida, mi inspiración y mi guía. Aunque ya no me acompañe en lo terrenal, sus enseñanzas me impulsan a seguir luchando por mis metas y sueños como el primer día. Este logro es por ustedes, mamá y papá, y lleva una parte de todos ustedes.

Resumen

Título: Implementación y análisis del desempeño de un pipeline para el estudio de amplicones 16S en un clúster energéticamente eficiente basado en Raspberry Pi 5.

Autores: Cristian Alberto Solano Torres; Zamir Francisco Granados Peñaloza.

Palabras Clave: 16S rRNA; pipelines bioinformáticos; Raspberry Pi 5; ARM64; eficiencia energética; clúster de bajo consumo.

El análisis bioinformático de amplicones del gen 16S rRNA es una herramienta ampliamente utilizada en estudios de diversidad microbiana, genómica ambiental y diagnóstico molecular. Sin embargo, sus etapas de control de calidad, filtrado, inferencia de variantes y asignación taxonómica se ejecutan tradicionalmente sobre estaciones de trabajo o sistemas de computación de alto rendimiento (HPC) con un elevado consumo energético, lo que limita su uso en lugares con infraestructura reducida. En este trabajo se diseñó y construyó un clúster basado en microcomputadores Raspberry Pi 5 con arquitectura ARM64 y sistema operativo Ubuntu Server 24.04, sobre el cual se portaron y ejecutaron tres pipelines bioinformáticos (Kraken 2, LotuS3 y QIIME 2) para el análisis de amplicones 16S. El desempeño energético y computacional del clúster se comparó con el de dos computadores portátiles de arquitectura x86 con el mismo procesador, uno con Ubuntu 24.04 LTS nativo y otro con Ubuntu 22.04 LTS sobre el subsistema Windows Subsystem for Linux (WSL2), midiendo el tiempo de ejecución, el uso de CPU y memoria RAM y el consumo energético, a partir de los cuales se calculó la eficiencia energética expresada en lecturas procesadas (reads) por vatio-hora (reads/Wh). Los resultados demostraron que los tres pipelines se ejecutaron exitosamente en el clúster ARM64 y preservaron la concordancia taxonómica frente a la plataforma x86 de referencia: en Kraken 2 y QIIME 2 las tablas de asignación taxonómica fueron idénticas bit a bit, y en LotuS3 las tres plataformas detectaron las mismas familias y los mismos géneros,

con la misma puntuación F1 (F1-score). En cuanto al tiempo de ejecución, con las muestras repartidas en cinco nodos de cómputo el clúster fue 2.58× más rápido que la estación x86 nativa en QIIME 2, en LotuS3 se obtuvo un desempeño similar (0.96×) y en Kraken 2 la estación x86 nativa fue 2.32× más rápida. Cabe resaltar que en Kraken2 y LotuS3 el tiempo del clúster quedó limitado por el almacenamiento de red compartido, desde el cual los cinco nodos leían la base de referencia al mismo tiempo. En términos de consumo energético, el clúster fue más eficiente que la estación x86 nativa en energía neta: 3.81× en Kraken 2, 2.14× en LotuS3 y 4.57× en QIIME 2. Estos factores son resultados observados con el protocolo de medición adoptado (PMIC a nivel de placa en el clúster frente a RAPL del encapsulado en x86) y no constituyen equivalencias generales entre arquitecturas. Como delimitaciones y limitaciones generales del proyecto se identifican las siguientes: Todos los experimentos se realizaron con secuencias sintéticas simuladas por la plataforma LEMMI16s para tener control sobre los resultados biológicos (*Ground Truth*). Mientras que Kraken2 y QIIME2 operaron con sus clasificadores nativos basados en k-meros y Naive Bayes respectivamente, en LotuS3 la asignación taxonómica se realizó mediante el software VSEARCH para asegurar la reproducibilidad cruzada frente a la arquitectura ARM64. La memoria física disponible (16 GB por nodo) obligó a acotar la base taxonómica SILVA en QIIME 2 a las regiones V1–V2 para prevenir desbordamientos de memoria (OOM), descartando la clasificación contra bases globales completas sin reducción. El enlace Gigabit Ethernet y el almacenamiento NFS compartidos impusieron cuellos de botella de I/O en tareas de alta transferencia, como en Kraken 2 y LotuS3. El aporte de este trabajo no está en construir un clúster de Raspberry Pi, sino en portar y validar pipelines de análisis de amplicones 16S en ARM64, demostrar la equivalencia de sus resultados biológicos frente a x86 y caracterizar experimentalmente el equilibrio entre rendimiento y consumo energético en estas cargas bioinformáticas.

Abstract

Title: Implementation and performance analysis of a pipeline for 16S amplicon studies on an energy-efficient cluster based on Raspberry Pi 5.

Authors: Cristian Alberto Solano Torres; Zamir Francisco Granados Peñaloza.

Keywords: 16S rRNA; bioinformatics pipelines; Raspberry Pi 5; ARM64; energy efficiency; low-power cluster.

Bioinformatic analysis of 16S rRNA gene amplicons is a widely used tool in microbial diversity studies, environmental genomics, and molecular diagnostics. However, its quality control, filtering, variant inference, and taxonomic assignment stages are traditionally executed on workstations or high-performance computing (HPC) systems with high energy consumption, which limits their use in locations with reduced infrastructure. In this work, a cluster based on Raspberry Pi 5 single-board computers with ARM64 architecture and Ubuntu Server 24.04 operating system was designed and built, on which three bioinformatic pipelines (Kraken 2, LotuS3, and QIIME 2) were ported and executed for 16S amplicon analysis. The energetic and computational performance of the cluster was compared with that of two x86 architecture laptop computers with the same processor, one with native Ubuntu 24.04 LTS and the other with Ubuntu 22.04 LTS on the Windows Subsystem for Linux (WSL2), measuring execution time, CPU and RAM usage, and energy consumption, from which energy efficiency was calculated, expressed in processed reads (reads) per watt-hour (reads/Wh). The results demonstrated that the three pipelines executed successfully on the ARM64 cluster and preserved taxonomic concordance compared to the reference x86 platform: in Kraken 2 and QIIME 2 the taxonomic assignment tables were bit-for-bit identical, and in LotuS3 the three platforms detected the same families and the same genera, with the same F1 score (F1-score). Regarding execution time, with the samples distributed across five compute

nodes, the cluster was 2.58× faster than the native x86 station in QIIME 2, in LotuS3 a similar performance was obtained (0.96×), and in Kraken 2 the native x86 station was 2.32× faster. It should be noted that in Kraken 2 and LotuS3, the cluster's time was limited by the shared network storage, from which the five nodes read the reference database at the same time. In terms of energy consumption, the cluster was more efficient than the native x86 station in net energy: 3.81× in Kraken 2, 2.14× in LotuS3, and 4.57× in QIIME 2. These factors are results observed under the adopted measurement protocol (board-level PMIC on the cluster versus package-level RAPL on x86) and do not constitute general equivalences between architectures. As general delimitations and limitations of the project, the following are identified. All experiments were performed with synthetic sequences simulated by the LEMMI16s platform to have control over the biological results (*Ground Truth*). While Kraken 2 and QIIME 2 operated with their native classifiers based on k-mers and Naive Bayes respectively, in LotuS3 taxonomic assignment was performed using the VSEARCH software to ensure cross-reproducibility against the ARM64 architecture. The available physical memory (16 GB per node) required restricting the SILVA taxonomic database in QIIME 2 to the V1–V2 regions to prevent out-of-memory (OOM) errors, ruling out classification against complete global databases without reduction. The shared Gigabit Ethernet link and NFS storage imposed I/O bottlenecks in high-transfer tasks, as in Kraken 2 and LotuS3. The contribution of this work does not lie in building a Raspberry Pi cluster, but in porting and validating 16S amplicon analysis pipelines on ARM64, demonstrating the equivalence of their biological results with x86, and experimentally characterizing the trade-off between performance and energy consumption for these bioinformatic workloads.

Índice general

Agradecimientos **1**

Resumen **3**

Abstract **5**

Glosario **12**

Introducción **14**

Planteamiento y justificación del problema 15

Objetivos 16

1. Marco de referencia **18**

1.1. Marco teórico 18

1.1.1. El marcador 16S rRNA 18

1.1.2. Pipelines bioinformáticos para el análisis de amplicones 16S 19

1.1.3. Recursos computacionales requeridos para el análisis de amplicones 16S 20

1.1.4. Clústeres de bajo consumo energético 21

1.1.5. Raspberry Pi 5 y arquitectura ARM64 22

1.2. Estado del arte 23

1.2.1. Clústeres de bajo costo y bajo consumo 23

1.2.2. Pipelines de análisis de amplicones 16S en arquitecturas alternativas (GPU, FPGA, RISC-V) 25

2. Metodología **27**

2.1. Requerimientos, restricciones y criterios de aceptación 28

2.2. Entornos experimentales 28

2.3. Fase 1: Diseño de la arquitectura hardware y software del clúster 31

2.4. Fase 2: Portabilidad de los pipelines a ARM64 36

2.5. Fase 3: Validación de la integridad funcional 40
2.6. Fase 4: Evaluación de desempeño y eficiencia energética 42

3. Resultados **46**
3.1. Fase 1: Clúster RBPi5 y línea base energética 46
3.2. Fase 2: Pipelines portados a ARM64 48
3.3. Fase 3: Validación de resultados biológicos 49
3.4. Fase 4: Desempeño computacional y eficiencia energética 51
3.4.1. Ejecución secuencial frente a distribuida 51
3.4.2. Análisis de desempeño y perfiles de potencia: Kraken 2 + Bracken . . 55
3.4.3. Análisis de desempeño y perfiles de potencia: LotuS3 Suite 58
3.4.4. Análisis de desempeño y perfiles de potencia: QIIME 2 62
3.4.5. Eficiencia comparativa global y balance de Green HPC 66

4. Conclusiones **68**
4.1. Conclusiones generales 68
4.2. Limitaciones del estudio 70
4.3. Impacto científico y aplicaciones prácticas 70
4.4. Trabajo futuro 71

Repositorio y recursos complementarios **77**

Índice de tablas

2.1. Requerimientos, restricciones, decisiones de diseño y criterios de aceptación del sistema. 29
2.2. Plataformas de cómputo utilizadas en la evaluación. 31

3.1. Potencia en reposo (*stand-by*) de los nodos de cómputo del clúster RBPi5, promedio de las mediciones de 5 minutos previas a cada corrida. 47
3.2. Pipelines portados a la arquitectura ARM64. 48
3.3. Comparación de resultados biológicos y taxonómicos frente al *Ground Truth*. 49
3.4. Tiempo y energía acumulada por plataforma: laptops x86 y clúster bajo ejecución secuencial y distribuida. 52
3.5. Métricas de desempeño computacional y energético: Kraken 2 + Bracken (5 muestras, 2,453,431 lecturas, protocolo 5 min de reposo basal). 55
3.6. Métricas de desempeño computacional y energético: LotuS3 Suite (5 muestras, 25,611 lecturas PacBio HiFi, protocolo 5 min de reposo basal). 58
3.7. Métricas de desempeño computacional y energético: QIIME 2 (5 muestras, 2,690,077 lecturas, protocolo 5 min de reposo basal). 62
3.8. Eficiencia energética comparativa en lecturas biológicas procesadas por vatiohora (*Reads/Wh*) bajo protocolo unificado de 5 min de reposo. 66

Índice de figuras

1.1. Estructura del RNA ribosomal 16S 18
1.2. Clúster de Raspberry Pi 22
1.3. Tarjeta Raspberry Pi 5 23

2.1. Ciclo DMAIC de la metodología Six Sigma 27
2.2. Entornos de experimentación: clúster RBPi5 con su topología de red, laptop x86 con Ubuntu nativo y laptop x86 con WSL2. Fuente: Propia. 30

3.1. Estado de los cinco nodos de cómputo en la partición `partition1` de Slurm. Fuente: Propia. 46
3.2. Clúster RBPi5 en operación: nodo maestro Raspberry Pi 4B (caja roja), switch Gigabit Ethernet y dos torres de cinco nodos de cómputo Raspberry Pi 5. Fuente: Propia. 47
3.3. Arquitectura y versiones de los pipelines en el nodo de cómputo `rbp5-1` del clúster. Fuente: Propia. 49
3.4. Fidelidad taxonómica (F1-Score) frente al *Ground Truth*: comparación de reproducibilidad biológica entre las estaciones x86_64 (Ubuntu nativo y WSL2) y el clúster ARM64. 50
3.5. Tiempo de ejecución por plataforma, evaluando la ejecución secuencial y distribuida sobre 5 nodos. 52
3.6. Energía total y neta por plataforma, evaluando la ejecución secuencial y distribuida en el clúster. 54
3.7. Kraken 2 en el clúster: potencia activa de cada nodo durante la ejecución distribuida y línea base de reposo previo (la curva gris es el reposo de 5 minutos medido justo antes de cada corrida). 56
3.8. Kraken 2 en la laptop con WSL2: potencia durante el procesamiento secuencial de las 5 muestras y reposo previo (gris). 57

3.9. Kraken 2 en la laptop con Ubuntu nativo: potencia durante el procesamiento de las 5 muestras y reposo previo (gris). 57
3.10. Kraken 2: potencia normalizada ($P_{\text{activa}} - \bar{P}_{\text{idle}}$) comparativa entre las tres plataformas evaluadas. 58
3.11. LotuS3 en el clúster: promedio por minuto de la potencia de cada nodo y del reposo previo (gris). 60
3.12. LotuS3 en la laptop con WSL2: promedio por minuto de la potencia durante el procesamiento de las 5 muestras y del reposo previo (gris). 60
3.13. LotuS3 en la laptop con Ubuntu nativo: promedio por minuto de la potencia durante el procesamiento de las 5 muestras y del reposo previo (gris). 61
3.14. LotuS3: promedio por minuto de la potencia normalizada ($P_{\text{activa}} - \bar{P}_{\text{idle}}$) comparativa entre las tres plataformas evaluadas. 62
3.15. QIIME 2 en el clúster: promedio por minuto de la potencia de cada nodo y de la línea base de reposo (gris). 63
3.16. QIIME 2 en la laptop con WSL2: promedio por minuto de la potencia y de la línea base de reposo (gris). 64
3.17. QIIME 2 en la laptop con Ubuntu nativo: promedio por minuto de la potencia y de la línea base de reposo (gris). 64
3.18. QIIME 2: promedio por minuto de la potencia normalizada ($P_{\text{activa}} - \bar{P}_{\text{idle}}$) comparativa. 65
3.19. Eficiencia energética final ($\eta = N_{\text{reads}}/E$) expresada en lecturas procesadas por vatio-hora (*Reads/Wh*) en escala logarítmica. 66

Glosario

A continuación se definen los términos principales utilizados a lo largo del documento.

16S rRNA: gen que codifica el RNA ribosomal en procariotas. Gracias a sus regiones conservadas e hipervariables, actúa como el principal marcador para la identificación y clasificación de bacterias.

Amplicón: fragmento de DNA o RNA que es el producto directo de un evento de amplificación. Es la secuencia base para identificar la diversidad microbiana en este estudio.

ARM64: arquitectura de procesador de 64 bits basada en un conjunto reducido de instrucciones. Su principal ventaja frente a arquitecturas tradicionales es que ofrece un rendimiento competitivo con un consumo de energía significativamente menor.

ASV (*Amplicon Sequence Variant*): variante exacta de una secuencia de amplicón, inferida con precisión de un solo nucleótido. Permite distinguir microorganismos con mayor resolución que los métodos tradicionales de agrupamiento

Clúster: conjunto de computadores interconectados que trabajan de manera coordinada como si fueran un solo sistema. Permite distribuir tareas pesadas entre varios nodos para reducir el tiempo de procesamiento.

Green HPC (cómputo verde): enfoque de la computación de alto rendimiento que busca maximizar el trabajo realizado por unidad de energía consumida, priorizando hardware y configuraciones de bajo consumo eléctrico.

HPC (*High Performance Computing*): computación de alto rendimiento. Hace referencia a sistemas con gran capacidad de cómputo, como supercomputadores o servidores especializados, típicamente usados en investigación científica, pero con alto consumo energético.

***k*-mer:** fragmento de longitud fija extraído de una secuencia de DNA. Las herramientas bioinformáticas los usan para comparar secuencias rápidamente contra bases de datos

e identificar qué organismos están presentes en una muestra.

OTU (*Operational Taxonomic Unit*): agrupamiento de secuencias de DNA con alta similitud entre sí, usado como unidad práctica para representar especies o grupos de microorganismos en estudios de diversidad microbiana.

Pipeline bioinformático: sucesión automatizada de algoritmos y herramientas de software diseñadas para procesar datos biológicos crudos, filtrarlos y convertirlos en información analizable.

reads/Wh: métrica de eficiencia energética que indica cuántas lecturas de secuenciación procesa un sistema por cada vatio-hora de energía consumida. A mayor valor, más eficiente es el sistema.

SBC (*Single Board Computer*): computador completo contenido en una sola placa de circuito impreso. Son dispositivos compactos, económicos y de bajo consumo, como la Raspberry Pi 5 utilizada en este proyecto.

Six Sigma (DMAIC): metodología de mejora continua orientada a la reducción de variabilidad y la optimización del rendimiento, adaptada aquí para la validación y mejora de sistemas computacionales. Sus etapas son Definir, Medir, Analizar, Mejorar y Controlar.

WSL2 (*Windows Subsystem for Linux 2*): capa de compatibilidad de Windows que ejecuta un núcleo Linux real dentro de una máquina virtual ligera, permitiendo usar distribuciones como Ubuntu sin reemplazar el sistema operativo anfitrión.

Introducción

El gen 16S es un fragmento molecular del ácido desoxirribonucleico (DNA) utilizado para ayudar a identificar microorganismos, especialmente bacterias. La secuenciación de amplicones 16S es un método de laboratorio que usa pasos detallados y tecnologías específicas para capturarlo y representarlo en forma de archivo de texto, en formato FASTA o FASTQ (Cock, Fields, Goto, Heuer, y Rice, 2010). A partir de estos datos se realiza un análisis bioinformático que permite interpretar las secuencias obtenidas y que es ampliamente utilizado en estudios de diversidad microbiana, genómica ambiental y diagnóstico molecular (Clarridge, 2004). Este tipo de análisis contiene diversas etapas, como control de calidad, filtrado de secuencias, inferencia de variantes y asignación taxonómica, las cuales requieren arquitecturas multinúcleo y multitarea, como estaciones de trabajo o sistemas HPC, que presentan un elevado consumo de energía. Por ejemplo, una estación de trabajo científica puede llegar a consumir entre 300 y 600 W, y un nodo de computación de alto rendimiento entre 300 y 5000 W (Cox y cols., 2014). Estas restricciones dificultan el uso de dichas arquitecturas en lugares con infraestructura reducida y generan problemas de sostenibilidad energética.

Gracias al avance tecnológico, actualmente es posible disponer de sistemas de cómputo basados en microordenadores, que permiten construir sistemas con menores prestaciones computacionales en comparación con una estación de trabajo o un sistema HPC, pero con un menor consumo energético (Pyrhönen y Ikonen, 2025). En este proyecto se analizó el consumo energético de pipelines de análisis de amplicones 16S ejecutados sobre un clúster de computación basado en microcomputadores Raspberry Pi 5 (en adelante RBPi5). La elección de esta plataforma se justifica por su arquitectura ARM de 64 bits, su bajo consumo energético y su capacidad para soportar la ejecución de programas concurrentes. Los reportes de fábrica indican que, la RBPi5 se alimenta con una fuente de 5 V/5 A, equivalente a una potencia de 25 W por unidad (Raspberry Pi Ltd., 2023).

El objetivo de este proyecto fue implementar y evaluar pipelines bioinformáticos para el análisis de amplicones 16S, orientando su ejecución hacia infraestructuras más eficientes

energéticamente. Para lograrlo, se construyó un clúster RBPi5 con Ubuntu Server 24.04 sobre el cual se portaron y ejecutaron tres pipelines, y se evaluó su comportamiento frente a dos computadores portátiles de arquitectura x86 (Intel) mediante métricas de tiempo de ejecución, demanda de recursos computacionales (CPU y RAM) y consumo energético, con el fin de analizar el balance entre desempeño y eficiencia energética, evaluada como el total de secuencias (*reads*) analizadas por unidad de energía (reads/Wh).

El documento se organiza de la siguiente manera: el Capítulo 1 presenta el marco de referencia, que incluye los fundamentos teóricos y el estado del arte. El Capítulo 2 describe la metodología y los entornos experimentales. El Capítulo 3 presenta los resultados obtenidos en cada fase. Y el Capítulo 4 expone las conclusiones y el trabajo futuro.

Planteamiento y justificación del problema

Los pipelines bioinformáticos para el análisis de amplicones 16S demandan recursos computacionales considerables. Herramientas como Kraken 2 (D. Wood, 2024; D. E. Wood, Lu, y Langmead, 2019) pueden requerir entre 8 y 100 GB de memoria RAM según la base de datos empleada, mientras que DADA2 y QIIME 2 requieren entre 8 y 32 GB para conjuntos de datos moderados (Bolyen y cols., 2019; Callahan y cols., 2016). A esto se suma la necesidad de almacenamiento para archivos FASTQ y bases de datos que pueden ocupar decenas o cientos de gigabytes. Por esta razón, su ejecución se ha concentrado tradicionalmente en infraestructuras robustas, como estaciones de trabajo multinúcleo y sistemas HPC (Seppey, Benavides, Berkeley, Manni, y Zdobnov, 2025), cuyo consumo eléctrico es elevado.

Las alternativas de aceleración por hardware especializado, como GPU y FPGA, reducen los tiempos de procesamiento, pero su alto costo, su consumo energético y la complejidad de su configuración dificultan su adopción en entornos con recursos limitados (Robinson, Harkin, y Shukla, 2021). Por otro lado, aunque los clústeres basados en arquitectura ARM han demostrado ser eficientes para diversas cargas científicas (Maqbool, Oh, y Fox, 2015; Warade, Schneider, y Lee, 2022), en la literatura revisada no se encontraron reportes que validen la operatividad o el desempeño de la mayoría de los pipelines de análisis de amplicones 16S sobre arquitectura ARM64, y las implementaciones existentes en dispositivos de bajo consumo, como las de EPI2ME Labs, están orientadas principalmente al análisis metagenómico restringidos a tecnologías Oxford Nanopore (Oxford Nanopore Technologies, 2024).

En consecuencia, se hace necesario explorar plataformas accesibles, de bajo consumo y fácil implementación que puedan ejecutar análisis bioinformáticos de amplicones 16S, y cuantificar objetivamente su desempeño y eficiencia energética en plataformas de bajo consumo

energético frente a las arquitecturas convencionales. Este proyecto muestra la implementación y evaluación de pipelines bioinformáticos en un clúster RBPi5, plataforma que combina la arquitectura ARM64 con un bajo consumo de energía y un costo mucho menor que el de las alternativas.

El proyecto se abordó como un problema de ingeniería: a partir de esta necesidad se definieron los requerimientos del sistema, las restricciones impuestas por el hardware ARM64 y por la infraestructura compartida, las decisiones de diseño y los criterios de aceptación con los que se verificó cada fase (Sección 2.1). Su aporte no está en construir un clúster de Raspberry Pi ni en mostrar que ARM consume menos energía, aspectos que ya tienen antecedentes en la literatura (Sección 1.2), sino en portar y validar pipelines de análisis de amplicones 16S en ARM64, demostrar la equivalencia de sus resultados biológicos frente a x86 y caracterizar experimentalmente el equilibrio entre rendimiento y consumo energético en estas cargas bioinformáticas.

En este contexto, el proyecto responde la siguiente pregunta de investigación:

¿Cuál es la eficiencia energética y el rendimiento de procesamiento de un pipeline de análisis de amplicones 16S ejecutado en un clúster Raspberry Pi 5, frente a una estación de trabajo de arquitectura x86 estándar (total de secuencias analizadas por unidad de energía, reads/Wh)?

Objetivos

Objetivo General

Analizar el desempeño y la eficiencia energética de un pipeline bioinformático para el estudio de amplicones 16S, ejecutado sobre un clúster de Raspberry Pi 5 frente a una estación de trabajo de arquitectura x86 estándar.

Objetivos Específicos

1. Diseñar la arquitectura hardware y software del clúster, definiendo la topología de red y los componentes necesarios para el despliegue del sistema bioinformático sobre arquitectura ARM64.
2. Portar a la arquitectura ARM64 un pipeline bioinformático de análisis de 16S, permitiendo su ejecución en paralelo dentro del clúster.

3. Validar la integridad funcional del pipeline mediante el procesamiento de conjuntos de datos de amplicones representativos, verificando que los resultados biológicos obtenidos en el clúster Raspberry Pi 5 sean consistentes y comparables con los generados en una arquitectura x86 estándar o similar.
4. Evaluar el desempeño del sistema comparando métricas de tiempo de ejecución, uso de recursos (CPU y memoria) y consumo energético frente a una estación de trabajo de control, estableciendo la relación de eficiencia final en reads/Wh.

Capítulo 1

Marco de referencia

1.1. Marco teórico

1.1.1. El marcador 16S rRNA

El gen 16S rRNA (ácido ribonucleico ribosómico), mostrado en la Figura 1.1.1, es parte del DNA ribosómico y se encuentra en todas las bacterias y arqueas. Se emplea extensamente como un marcador filogenético universal. Tiene una longitud aproximada de 1.500 pares de bases y presenta zonas con un alto grado de conservación, lo cual simplifica el diseño de marcadores universales. Entre estas zonas hay otras regiones llamadas hipervariables (V1 a V9), que tienen la suficiente variabilidad para permitir la distinción entre linajes de bacterias (Clarridge, 2004). Todas estas características convierten al 16S rRNA en un marcador útil para estudiar la diversidad bacteriana y establecer relaciones filogenéticas entre microorganismos.

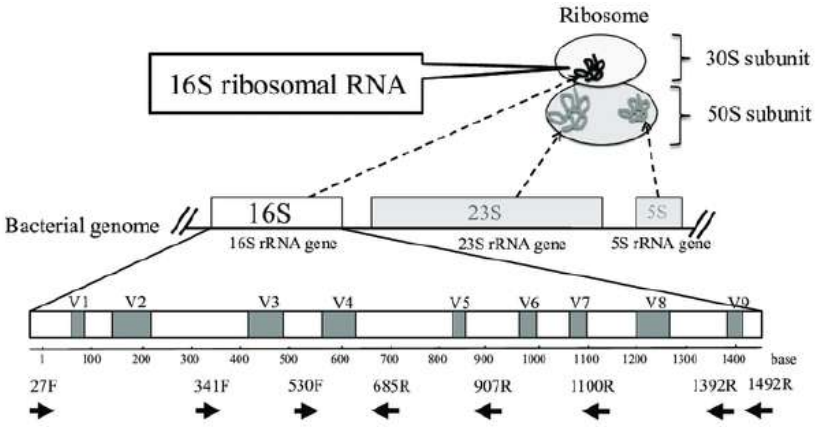


Figura 1.1: Estructura del RNA ribosomal 16S. Fuente: Fukuda y cols. (2016).

1.1.1.1. Aplicaciones del gen 16S en la caracterización bacteriana

La caracterización bacteriana es clave para identificar y entender con precisión cómo funcionan los microorganismos en distintos entornos. Una de las aplicaciones más importantes del gen 16S está en la microbiología clínica, donde permite detectar bacterias patógenas en muestras biológicas (Clarridge, 2004). En la ecología microbiana, esta técnica sirve para mapear comunidades bacterianas en ecosistemas diversos. En Colombia, por ejemplo, se ha utilizado para estudiar la composición microbiana de aguas termales en la región Andina (Benavides, Isaza, Niño-García, Alzate, y Cabarcas, 2018). El gen 16S también es fundamental para documentar cómo la actividad humana altera la diversidad microbiana en fuentes de agua (Zhang, Wang, Li, Chen, y Liu, 2025) y para potenciar procesos de biorremediación en sedimentos marinos (Lee, Jeong, Kim, y Kim, 2021). Además, se usa en el análisis del microbioma intestinal, permitiendo vincular ciertas proliferaciones bacterianas con enfermedades inflamatorias y problemas digestivos (Sehli, Idrissi Azami, y Dini, 2022).

1.1.2. Pipelines bioinformáticos para el análisis de amplicones 16S

Los pipelines bioinformáticos son procesos computacionales para el estudio de los amplicones 16S. Sus etapas incluyen, a nivel general, el control de calidad de lecturas (*reads*), el recorte de adaptadores y cebadores, el agrupamiento o reducción de ruido en las secuencias, la clasificación taxonómica y el análisis de diversidad. En la literatura se han reportado diferentes pipelines que automatizan este proceso, entre los cuales sobresalen los siguientes.

- **Kraken 2** (D. E. Wood y cols., 2019) es una herramienta de clasificación taxonómica basada en k-mers (subsecuencias de longitud fija k que se obtienen al segmentar una lectura de DNA). Asigna las lecturas de secuenciación a taxones mediante la comparación contra bases de datos de referencia. Su propósito original fue la metagenómica *shotgun* (Quince, Walker, Simpson, Loman, y Segata, 2017). Sin embargo, su alta velocidad de procesamiento ha extendido su uso al análisis de amplicones 16S. Trabajos como los de EPI2ME Labs demuestran su viabilidad en dispositivos ARM64 como Jetson Nano o Raspberry Pi (Oxford Nanopore Technologies, 2024), aunque dichas implementaciones están orientadas a análisis metagenómicos.
- **LotuS3** (*Less OTU Scripts 3*) (Hildebrand y Özkurt, 2024) es un pipeline de código abierto que sucede a LotuS2 (Özkurt y cols., 2022) e incorpora etapas integradas de filtrado de calidad estricto, agrupamiento (*clustering*) en OTUs y clasificación taxonómica, con énfasis en la reproducibilidad y la eficiencia computacional, lo que lo hace relevante en contextos de evaluación comparativa (*benchmarking*) de herramientas pa-

ra el análisis de lecturas 16S. Su disponibilidad como software libre y su capacidad de ejecución nativa sobre arquitectura ARM64 lo posicionan como una opción viable para infraestructuras de bajo consumo. No obstante, la literatura carece de reportes sobre su caracterización energética en ecosistemas ARM64.

- **QIIME 2** (*Quantitative Insights Into Microbial Ecology 2*) (Bolyen y cols., 2019) es una plataforma modular de código abierto que integra múltiples herramientas para analizar lecturas 16S. Soporta tanto el agrupamiento en OTU (*Operational Taxonomic Units*) como la inferencia de ASV (*Amplicon Sequence Variants*) mediante complementos como DADA2 y Deblur, y ofrece capacidades de visualización interactiva de los resultados. En la revisión realizada a la fecha, no se hallaron registros que validen la operatividad o el desempeño de este pipeline sobre arquitectura ARM64.
- **DADA2** (*Divisive Amplicon Denoising Algorithm 2*) (Callahan y cols., 2016) es un método estadístico de reducción de ruido que modela los errores de secuenciación para inferir variantes de secuencia de amplicones (ASV) con resolución de un solo nucleótido. Está implementado en R, lo que permite su uso de forma independiente o integrado dentro de QIIME 2. Trabajos recientes de cómputo de alto rendimiento sobre procesadores ARM con memoria unificada demuestran reducciones en los cuellos de botella de algoritmos como `learnErrors` (Scientific Community and HPC Researchers, 2025). Sin embargo, aunque su diseño en R lo hace compatible con ARM64, a la fecha no se reportan trabajos específicos de su uso en esta arquitectura para amplicones 16S.
- **EMU** (*Expectation-Maximization for Uniquely identifying 16S*) (Curry y cols., 2022) es un pipeline orientado a la caracterización de comunidades microbianas a nivel de especie, desarrollado para procesar lecturas de longitud completa del gen 16S rRNA obtenidas mediante secuenciación Oxford Nanopore (Jain, Olsen, Paten, y Akeson, 2016). Su principal ventaja es la capacidad de aprovechar la longitud de las lecturas para lograr una mayor resolución taxonómica frente a tecnologías de lecturas cortas. A la fecha no existe evidencia documentada de su implementación o validación en arquitecturas ARM64.

1.1.3. Recursos computacionales requeridos para el análisis de amplicones 16S

La ejecución de pipelines bioinformáticos dirigidos al análisis de amplicones 16S requiere recursos computacionales cuya capacidad varía según la cantidad de datos del experimento, las herramientas utilizadas y la complejidad del análisis. En general, estos pipelines requieren

arquitecturas multinúcleo que permitan la simultaneidad de etapas como el control de calidad y la asignación taxonómica. Por ello es habitual el uso de estaciones de trabajo con múltiples núcleos o de sistemas HPC (Seppey y cols., 2025). A nivel de memoria RAM, herramientas como Kraken 2 llegan a requerir entre 8 y 100 GB según la base de datos de referencia empleada (D. Wood, 2024; D. E. Wood y cols., 2019), mientras que DADA2 y QIIME 2 requieren entre 8 y 32 GB para conjuntos de datos moderados (Bolyen y cols., 2019; Callahan y cols., 2016).

El almacenamiento también es fundamental, pues los archivos de secuenciación en formato FASTQ, junto con las bases de datos de referencia y los archivos intermedios del pipeline, pueden ocupar decenas o cientos de gigabytes. Este conjunto de requisitos explica el uso tradicional de infraestructuras robustas y de alto consumo energético para este tipo de análisis, y motiva la exploración de opciones energéticamente menos demandantes como la planteada en este proyecto.

1.1.4. Clústeres de bajo consumo energético

Estudios recientes han mostrado que los centros de datos y los sistemas HPC representan una gran proporción del consumo eléctrico global (Warade y cols., 2022), lo que ha impulsado el desarrollo de arquitecturas basadas en procesadores de bajo consumo, como ARM, para ejecutar cargas de trabajo científicas y de análisis de datos (Pyrhönen y Ikonen, 2025). En este contexto, la construcción de clústeres basados en RBPi5 surge como un modelo experimental para contrastar si la reducción en el consumo eléctrico de la arquitectura ARM64 permite alcanzar una eficiencia adecuada en el procesamiento de datos frente a la capacidad de cómputo de las arquitecturas x86.



Figura 1.2: Clúster de Raspberry Pi. Fuente: Propia.

Un clúster de bajo consumo energético, como el de la Figura 1.2, es un sistema de cómputo distribuido conformado por múltiples nodos interconectados mediante una red de comunicación, cuyo diseño prioriza la eficiencia energética como criterio fundamental de construcción y operación. A diferencia de los sistemas HPC tradicionales, que priorizan el rendimiento computacional a expensas de un elevado consumo eléctrico, los clústeres de bajo consumo buscan maximizar la relación entre capacidad de procesamiento y energía consumida, expresada comúnmente en métricas como operaciones de punto flotante por vatio (FLOPS/W) o, en el contexto bioinformático, lecturas procesadas por unidad de energía (reads/Wh) (Pyrhönen y Ikonen, 2025).

1.1.5. Raspberry Pi 5 y arquitectura ARM64

La Raspberry Pi 5, mostrada en la Figura 1.3, llegó al mercado en 2023 como la quinta generación de la línea de computadores de placa única (*Single Board Computer*, SBC) de Raspberry Pi. Cuenta con un procesador Broadcom BCM2712 de cuatro núcleos ARM Cortex-A76 de 64 bits (ARM64) a velocidades de hasta 2,4 GHz (Pyrhönen y Ikonen, 2025). A nivel de memoria, soporta hasta 16 GB de RAM LPDDR4X y, en cuanto a conectividad, ofrece puertos USB 3.0, interfaz PCIe 2.0, salida dual de video 4K y red Gigabit Ethernet (Raspberry Pi Ltd., 2023).



Figura 1.3: Visualización de la tarjeta Raspberry Pi 5. Fuente: Propia.

La arquitectura ARM (*Advanced RISC Machine*) de la RBPi5 se basa en el paradigma RISC (*Reduced Instruction Set Computing*), caracterizado por un conjunto reducido de instrucciones que permite alcanzar un alto rendimiento por vatio consumido (Pyrhönen y Ikonen, 2025). La arquitectura ARM64 introduce un conjunto ampliado de registros de propósito general y brinda soporte nativo para operaciones de punto flotante y extensiones SIMD (*Single Instruction, Multiple Data*) a través del conjunto de instrucciones NEON, las cuales son importantes para acelerar operaciones de procesamiento de datos biológicos como el alineamiento de secuencias (Pyrhönen y Ikonen, 2025). Gracias a esta configuración, un clúster basado en RBPi5 se presenta como una alternativa viable para desplegar los entornos bioinformáticos descritos anteriormente, siempre que estos sean adaptados a la arquitectura ARM64 y el análisis a un número limitado de muestras.

1.2. Estado del arte

1.2.1. Clústeres de bajo costo y bajo consumo

La idea de obtener capacidad de cómputo paralelo a partir de equipos de bajo costo tiene su origen en el proyecto Beowulf (Sterling y cols., 1995), que mostró que un conjunto de computadores personales unidos por una red Ethernet y con software libre podía ofrecer un rendimiento útil para el cálculo científico a una fracción del costo de un supercomputador.

Con la llegada de los computadores de placa única, este enfoque dio lugar a los miniclústeres o clústeres SBC, como Iridis-Pi (Cox y cols., 2014), que reunió 64 Raspberry Pi como plataforma de bajo costo para la demostración y la enseñanza. Paralelamente, el prototipo Tibidabo del proyecto europeo Mont-Blanc mostró que los procesadores ARM de dispositivos móviles podían ser competitivos en eficiencia energética para HPC, aunque limitados por la red y la memoria (Rajovic, Rico, Puzovic, Adeniyi-Jones, y Ramirez, 2014).

En la Universidad Industrial de Santander, el Centro de Supercomputación y Cálculo Científico (SC3UIS) puso en funcionamiento en 2012 el supercomputador GUANE-1, un clúster de 16 nodos con procesadores Intel Xeon y GPU NVIDIA Tesla construido en un proyecto conjunto con HP y NVIDIA, que se ha empleado, entre otras aplicaciones, en simulaciones para la industria del petróleo (Vanguardia, 2012). El mismo centro ha estudiado los milliclusters, clústeres de procesadores ARM de bajo consumo, con modelos de energía y estrategias de calendarización de tareas que buscan reducir a la vez el tiempo de ejecución y el consumo energético (Flórez Suárez, 2016).

La evolución de los computadores de placa única ha permitido que superen el ámbito educativo, consolidándose como componentes viables y eficientes en infraestructuras de cómputo científico.

Un caso destacado es el de Abrahamsson y cols. (2013), quienes exploraron la viabilidad de nubes de bajo costo conectando 300 microcomputadores SBC en la Universidad Libre de Bolzano. Los investigadores mostraron que las limitaciones individuales de cada dispositivo pueden mitigarse mediante el paralelismo, de modo que el clúster, trabajando de forma simultánea, logra un rendimiento que supera las capacidades individuales de sus componentes.

Otro ejemplo es el documentado por Miller (2017), sobre el uso de 750 microcomputadores en el Laboratorio Nacional de Los Álamos para la creación de entornos de prueba a gran escala. Esta arquitectura resultó significativamente más eficiente desde el punto de vista energético que un supercomputador tradicional, manteniendo las capacidades necesarias para la validación y el desarrollo de software complejo.

En cuanto al rendimiento en entornos científicos, Maqbool y cols. (2015) mostraron que la arquitectura ARM ofrece una escalabilidad superior a x86 en aplicaciones con alta demanda de entrada y salida (E/S), con resultados competitivos en ancho de banda y eficiencia energética. Asimismo, Warade y cols. (2022) validaron que el uso de clústeres de bajo consumo puede reducir el gasto energético total hasta en un 68 % mediante una gestión optimizada de recursos.

Teniendo en cuenta estos antecedentes, la Raspberry Pi 5 introducida a finales de 2023

representa una evolución técnica relevante: gracias a su procesador Broadcom BCM2712 con cuatro núcleos ARM Cortex-A76 a 2.4 GHz e interfaz PCIe, amplía notablemente las capacidades de procesamiento de placas de bajo consumo frente a generaciones previas, permitiendo abordar la ejecución de pipelines bioinformáticos de análisis de amplicones en esquemas paralelos y distribuidos (Raspberry Pi Ltd., 2023).

Estos antecedentes establecen que los clústeres SBC y los procesadores ARM son viables para el cómputo científico de bajo consumo. Por ello, la contribución de este proyecto no está en construir un clúster de Raspberry Pi, sino en portar y validar pipelines de análisis de amplicones 16S en ARM64, comprobar que sus resultados biológicos son equivalentes a los de x86 y caracterizar experimentalmente el equilibrio entre rendimiento y consumo energético en estas cargas bioinformáticas.

1.2.2. Pipelines de análisis de amplicones 16S en arquitecturas alternativas (GPU, FPGA, RISC-V)

El procesamiento de datos de 16S rRNA ha buscado históricamente la aceleración mediante hardware especializado para mitigar el cuello de botella que supone el volumen de lecturas. En arquitecturas basadas en unidades de procesamiento gráfico (GPU) se han desarrollado herramientas de clasificación taxonómica masiva que aprovechan el paralelismo de hilos para procesos de reducción de ruido y asignación de k-mers. La aceleración por GPU ha permitido que herramientas como Kraken 2 alcancen velocidades de procesamiento órdenes de magnitud superiores a las de una CPU estándar, especialmente en metagenómica *shotgun* y amplicones, aunque este incremento conlleva un consumo energético considerablemente mayor (Robinson y cols., 2021; D. E. Wood y cols., 2019).

Por otro lado, la industria ha adoptado arquitecturas basadas en matrices de puertas lógicas programables en campo (FPGA), que permiten acelerar el alineamiento de secuencias y el conteo de variantes directamente en hardware. A diferencia de las GPU, las FPGA logran reducir la latencia del pipeline con un menor consumo energético, ya que su diseño configurable permite adaptar los caminos de datos al algoritmo específico, evitando el desperdicio de energía en operaciones innecesarias (Robinson y cols., 2021).

Finalmente, la arquitectura de código abierto RISC-V (Gómez-Sánchez, Berral, Torrents, y Pérez, 2023) ha comenzado a explorarse como una alternativa de ultra bajo consumo para el análisis de secuencias. Proyectos de investigación en computación de alto rendimiento han evaluado el desempeño de algoritmos de alineamiento sobre núcleos RISC-V, mostrando que, aunque su ecosistema de software aún está madurando, su diseño modular es adecuado para

tareas bioinformáticas que requieren personalización a nivel de instrucción para mejorar la eficiencia energética (Gómez-Sánchez y cols., 2023; Pyrhönen y Ikonen, 2025).

A pesar de estos avances, el alto costo, el elevado consumo de energía y la complejidad de configuración dificultan el uso de estas arquitecturas en entornos con recursos limitados. Esto hace necesario explorar plataformas accesibles, de bajo consumo y fácil implementación, como el clúster RBPi5 propuesto en este trabajo.

Capítulo 2

Metodología

El desarrollo del proyecto se estructuró en cuatro fases: (1) diseño y construcción del clúster RBPi5, (2) portabilidad e instalación de los pipelines sobre arquitectura ARM64, (3) validación funcional mediante datos de referencia, y (4) evaluación comparativa de desempeño y eficiencia energética frente a una arquitectura x86. Cada fase se llevó a cabo bajo la metodología Six Sigma, concebida por Bill Smith en 1986 (Purdue University, 2021), aplicando el ciclo DMAIC (*Define, Measure, Analyze, Improve, Control*), cuyo esquema general se presenta en la Figura 2.1.



Figura 2.1: Ciclo DMAIC de la metodología Six Sigma. Fuente: generada con Gemini (versión 3).

La metodología DMAIC fue concebida originalmente para la mejora de procesos existentes. En este proyecto, dado que el objeto de estudio es la construcción e implementación de un sistema tecnológico nuevo, sus etapas se adaptaron de la siguiente manera:

- **Definir:** establece los requerimientos y criterios de éxito de cada fase.
- **Medir:** caracteriza el estado inicial del sistema y documenta sus especificaciones técnicas.
- **Analizar:** examina los datos recopilados en la etapa anterior para identificar restricciones, incompatibilidades o cuellos de botella que orientan las decisiones técnicas.
- **Mejorar:** ejecuta las acciones correctivas o de implementación derivadas del análisis previo.
- **Controlar:** verifica la estabilidad y reproducibilidad del sistema antes de avanzar a la siguiente fase.

2.1. Requerimientos, restricciones y criterios de aceptación

El clúster y la portabilidad de los pipelines se abordaron como un problema de ingeniería. A partir de la necesidad de ejecutar análisis de amplicones 16S con bajo costo y bajo consumo, se definieron los requerimientos del sistema, se identificaron las restricciones impuestas por el hardware ARM64 y por la infraestructura compartida, y se tomaron decisiones de diseño verificables mediante criterios de aceptación. La Tabla 2.1 resume esta relación; cada decisión se desarrolla en las fases descritas a continuación y su verificación se presenta en el Capítulo 3.

2.2. Entornos experimentales

Para la evaluación comparativa se definieron tres entornos experimentales que permiten contrastar la arquitectura ARM64 frente a x86, así como evaluar el impacto de la virtualización ligera sobre x86 mediante la ejecución en Linux nativo y bajo WSL2, tal como se sintetiza en la Tabla 2.2.

El primer entorno corresponde al clúster de bajo consumo (Green HPC), conformado por un nodo maestro Raspberry Pi 4 Model B (Broadcom BCM2711 de cuatro núcleos Cortex-A72 a 1.8 GHz y 8 GB de RAM), encargado de la orquestación y los servicios del sistema,

Tabla 2.1: Requerimientos, restricciones, decisiones de diseño y criterios de aceptación del sistema.

Requerimiento	**Restricción**	**Decisión de diseño**	**Criterio de aceptación**	**Resultado**
Ejecutar los tres pipelines en ARM64	Contenedores de LEMMI16s no portables; LAMBDA3 no disponible para ARM64	Instalación nativa con Conda para `aarch64`; VSEARCH en LotuS3; `usearch12` compilado	Las cinco muestras se procesan en los tres pipelines	3 de 3 pipelines
Resultados biológicos equivalentes a x86	Versiones de software distintas entre arquitecturas	Mismas versiones en las tres plataformas; mismas entradas verificadas con MD5	Tablas idénticas o mismas familias, géneros y F1	Cumplido
Operar con 16 GB de RAM por nodo	SILVA completa requiere entre 32 y 64 GB	SILVA reducida a V1–V2 (~62 000 secuencias); límites de memoria con `cgroups`	Ejecución sin desbordamiento de memoria (OOM)	Cumplido
Distribuir el trabajo entre nodos	Red Gigabit y almacenamiento NFS compartido	Una muestra por nodo con Slurm	Los cinco nodos completan sus muestras	QIIME 2: 5.00× de 1 a 5 nodos
Medir la energía de forma comparable	PMIC (placa completa) frente a RAPL (encapsulado)	Mismo protocolo a 1 Hz; energía total y neta	Reposo y ejecución registrados en todas las corridas	Cumplido
Operación estable de los nodos	Subtensión y temperatura bajo carga sostenida	Fuente oficial de 27 W por nodo; enfriamiento activo PWM	Sin reducción de frecuencia (*throttling*)	`get_throttled = 0x0`

y por los cinco nodos de cómputo Raspberry Pi 5 asignados al proyecto, de los diez disponibles sobre todo el clúster. Cada nodo de cómputo cuenta con un procesador Broadcom BCM2712 de cuatro núcleos ARM Cortex-A76 a 2.4 GHz (20 núcleos de cómputo en total), 16 GB de memoria RAM LPDDR4X-4267 (80 GB en total) y una tarjeta MicroSD local de 64 GB (Clase 10, UHS-I U3) para el arranque del sistema operativo. El clúster opera con Ubuntu Server 24.04 LTS en todos sus nodos, utiliza una topología de red en estrella sobre un conmutador Gigabit Ethernet (1000BASE-T) y comparte las bases de datos y los archivos FASTQ mediante un servidor NFSv4 alojado en el nodo maestro y montado en `/shared`. La distribución y el control de los trabajos paralelos se gestionan con Slurm Workload Manager (versión 23.11), y la telemetría energética se toma a 1.0 Hz directamente de los rieles de alimentación de cada placa mediante el PMIC Renesas DA9091. La Figura 2.2 resume la topología del clúster y las características de los tres entornos de experimentación.

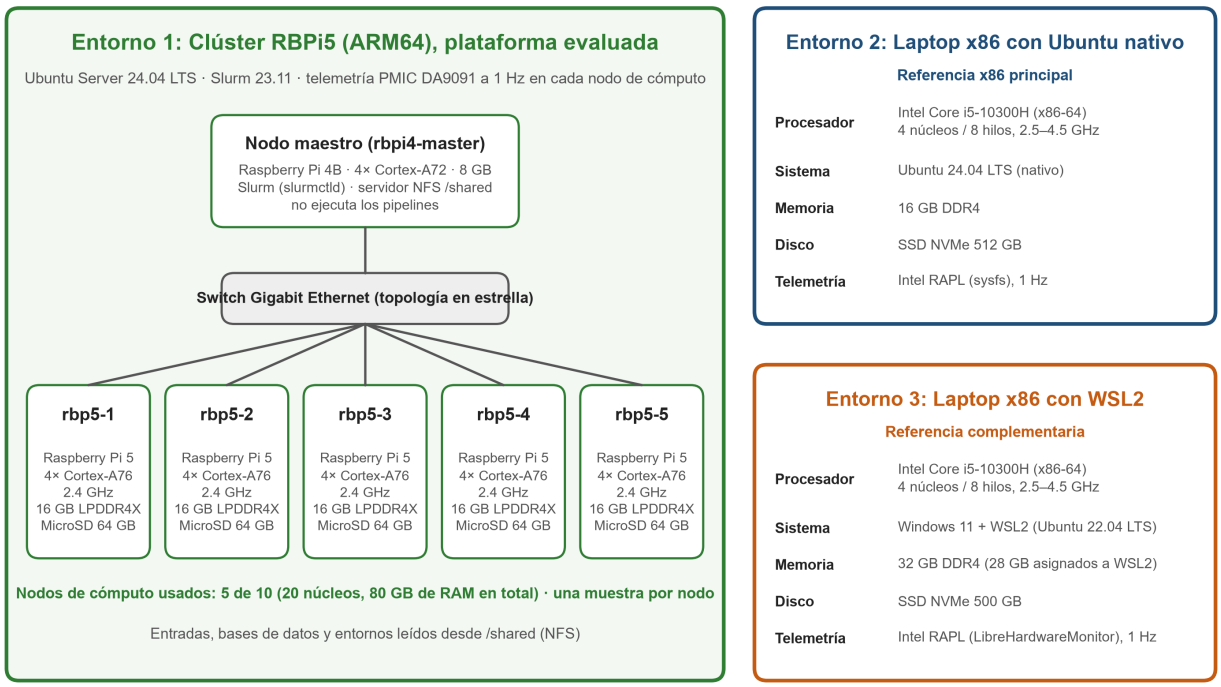


Figura 2.2: Entornos de experimentación: clúster RBPi5 con su topología de red, laptop x86 con Ubuntu nativo y laptop x86 con WSL2. Fuente: Propia.

El segundo entorno, utilizado como plataforma de referencia x86 principal, es un computador portátil con instalación nativa de Ubuntu 24.04 LTS (kernel Linux 6.8). Está equipado con un procesador Intel Core i5-10300H (4 núcleos físicos y 8 hilos lógicos a 2.50 GHz base, hasta 4.50 GHz turbo y 8 MB de caché), 16 GB de memoria RAM DDR4 a 2933 MHz en doble canal y una unidad de estado sólido SSD NVMe M.2 PCIe de 512 GB, lo que garantiza acceso directo al almacenamiento local sin sobrecargas de red. El consumo de potencia se registra a 1.0 Hz mediante la interfaz Intel RAPL en el dominio `package-0` a través de `sysfs`.

El tercer entorno, empleado como referencia complementaria, evalúa la ejecución virtualizada sobre un computador portátil con sistema anfitrión Windows 11 ejecutando la distribución invitada Ubuntu 22.04 LTS dentro de WSL2. Desde la máquina virtual de WSL2 son visibles los 4 núcleos y 8 hilos del procesador Intel Core i5-10300H (2.50 GHz), y se le asignaron dinámicamente 28 GB de memoria RAM mediante el archivo de configuración `.wslconfig` (de los 32 GB de RAM DDR4 disponibles). A nivel de disco, WSL2 opera sobre una unidad virtual en formato VHDX montada sobre un SSD NVMe de 500 GB, con sistema de archivos ext4, donde se almacenaron los datos y las bases de referencia. Su telemetría ener-

gética se registró a 1.0 Hz capturando los contadores RAPL de la CPU mediante el servicio LibreHardwareMonitor en Windows.

Tabla 2.2: Plataformas de cómputo utilizadas en la evaluación.

Plataforma	**Arquitectura (CPU)**	**RAM y Almacenamiento**	**Sistema operativo**	**Telemetría energética**	**Rol en el estudio**
Clúster RBPi5	ARM64 (20 núcleos Cortex-A76 @ 2.4 GHz)	80 GB LPDDR4X (total), MicroSD 64 GB, NFS compartido	Ubuntu Server 24.04 LTS (Slurm 23.11)	PMIC DA9091 (hardware ADC) a 1.0 Hz	Plataforma evaluada (Green HPC)
Computador x86 nativo	x86-64 (Intel Core i5-10300H)	16 GB DDR4, SSD NVMe 512 GB	Ubuntu 24.04 LTS nativo	MSR/RAPL (`sysfs`) a 1.0 Hz	Referencia x86 principal
Computador x86 WSL2	x86-64 (Intel Core i5-10300H)	32 GB DDR4 (28 GB para WSL2), SSD NVMe 500 GB	Ubuntu 22.04 LTS sobre WSL2 (Windows 11)	RAPL (LibreHardwareMonitor) a 1.0 Hz	Referencia complementaria

La articulación de estos tres entornos permite desacoplar y evaluar dos factores fundamentales: el impacto de la microarquitectura de bajo consumo (ARM64 frente a x86) y la sobrecarga computacional inducida por la virtualización ligera (Linux nativo frente a WSL2). No obstante, es indispensable destacar una delimitación metodológica en la instrumentación: mientras que en el clúster Raspberry Pi 5 la telemetría del PMIC captura el consumo a nivel de placa completa, en las plataformas x86 la interfaz Intel RAPL registra exclusivamente el dominio del encapsulado (`package-0`). Esta asimetría física se consideró explícitamente en el protocolo de experimentación, orientando la evaluación tanto a la energía bruta como a la neta para garantizar una comparación rigurosa y transparente entre sistemas.

2.3. Fase 1: Diseño de la arquitectura hardware y software del clúster

En esta fase se estructuró e implementó la arquitectura hardware y software del clúster de bajo consumo (Green HPC), compuesto por diez nodos de cómputo homogéneos Raspberry Pi 5 y un nodo maestro Raspberry Pi 4, definiendo la topología de red, el esquema de interconexión, la distribución de almacenamiento compartido y el sistema de telemetría energética para establecer la línea base de consumo del sistema en reposo (*stand-by*) sobre arquitectura ARM64.

Definir. Se formalizaron los requerimientos técnicos y funcionales del clúster para soportar cargas bioinformáticas altamente paralelas de procesamiento de secuencias masivas de amplicones 16S. A nivel de hardware, el clúster está organizado bajo un modelo maestro-trabajador y cuenta con un nodo maestro Raspberry Pi 4 Model B (`rbpi4-master`) y diez

nodos de cómputo Raspberry Pi 5. Para este proyecto se utilizaron cinco de ellos (`rbp5-1` a `rbp5-5`), que suman 20 núcleos físicos ARM Cortex-A76 @ 2.4 GHz y 80 GB de memoria RAM LPDDR4X (16 GB por nodo de cómputo).

A nivel de conectividad, se estableció una topología en estrella estructurada sobre un switch de red no administrado Gigabit Ethernet (1000BASE-T, IEEE 802.3ab) mediante cableado UTP Categoría 6, con un esquema de direccionamiento IPv4 estático privado dentro del segmento `192.168.1.100/24` a `192.168.1.105/24` y resolución local de nombres en `/etc/hosts`. El acceso al clúster se realizó de forma remota mediante SSH al nodo maestro, autenticándose con usuario y contraseña. Desde allí se enviaron los trabajos a los nodos de cómputo a través de Slurm (Versión 23.11.4). En cuanto a la infraestructura eléctrica, cada uno de los cinco nodos se dotó de una fuente de alimentación oficial Raspberry Pi USB-C de 27 W (5.1 V / 5.0 A con perfil Power Delivery), garantizando una entrega de corriente estable frente a picos transitorios de demanda de cómputo y evitando el estrangulamiento de frecuencia del microprocesador por subtensión (*undervoltage throttling*). Finalmente, se definió el estado de reposo (*stand-by*) como la condición operativa en la cual el nodo maestro y los cinco nodos de cómputo utilizados se encuentran completamente ensamblados, encendidos, conectados a la red local y con el kernel de Linux activo en modo multiusuario, pero sin procesos de análisis bioinformático en ejecución ni tráfico de datos en el bus de red.

Medir. Para poder comparar el consumo de las plataformas, primero se levantó un inventario del hardware de cada equipo y se definió cómo se iba a medir la energía en cada una de ellas.

En la plataforma ARM64, la medición se tomó directamente de la placa mediante el circuito integrado de gestión de energía (PMIC, *Power Management Integrated Circuit*) Renesas DA9091 disponible en cada Raspberry Pi 5. Este chip incluye un convertidor analógico-digital (ADC) de 12 canales que reporta el voltaje y la corriente de los rieles de alimentación de la placa. Estos valores se leyeron en cada nodo con la utilidad de firmware `vcgencmd pmic_read_adc`, llamada desde un script en Python de autoría propia (`medir_energia_pmic.py`) desarrollado específicamente para este proyecto, el cual captura los valores de los rieles a 1.0 Hz, calcula la potencia instantánea sumando el producto voltaje–corriente de todos los canales ($P = \sum V_i \cdot I_i$) y realiza la integración numérica para computar la energía consumida en vatios-hora.

En las plataformas x86, la telemetría se fundamentó en la interfaz Intel RAPL (*Running Average Power Limit*) para capturar el consumo del encapsulado del procesador en el dominio `package-0`, adaptando el mecanismo de recolección según el entorno del sistema operativo.

En *Ubuntu nativo* se empleó un script propio en Python (`medir_energia_rapl.py`) que consulta a 1.0 Hz los contadores de energía en microjulios expuestos por el kernel de Linux a través del pseudo-sistema de archivos `sysfs` (`/sys/class/powercap/intel-rapl:0/energy_uj`).

En *WSL2 sobre Windows 11* dado que el hipervisor de virtualización de WSL2 no expone los registros MSR/RAPL en el sistema de archivos de la máquina invitada de Linux, la medición se realizó desde el sistema anfitrión Windows mediante un script automatizado en PowerShell acoplado al motor de telemetría de *LibreHardwareMonitor*, registrando a 1.0 Hz el sensor de potencia del procesador (*CPU Package Power*) de forma sincronizada con la ejecución del pipeline.

Para obtener la línea base energética, en todas las plataformas se registró la potencia de cada equipo en reposo durante 5 minutos antes de cada corrida experimental tras un período de 60 segundos de estabilización previa sin medición, garantizando que únicamente los procesos fundamentales del sistema estuvieran en ejecución antes de iniciar el pipeline objetivo.

Analizar. A partir de los requerimientos identificados y los mecanismos de medición formulados, se llevó a cabo un análisis de viabilidad técnica y condiciones operativas enfocado en cuatro factores críticos de diseño:

En primer lugar, se analizó la capacidad de la infraestructura de comunicaciones frente a la demanda de datos bioinformáticos. Puesto que los pipelines requieren transferir concurrentemente conjuntos de lecturas FASTQ e índices taxonómicos desde el almacenamiento compartido NFS, se contrastó la tasa teórica del enlace Gigabit Ethernet (1000BASE-T, 125 MB/s) frente a los tamaños típicos de las muestras. Se determinó que una topología en estrella con resolución estática minimiza la latencia de conmutación y permite una distribución paralela eficiente, previendo que la lectura intensiva de bases de datos se beneficie de los búferes de memoria RAM local de cada nodo para descongestionar el tráfico de red.

En segundo lugar, se examinó el balance de potencia eléctrica y estabilidad de voltaje. Las especificaciones del microprocesador Broadcom BCM2712 operando a 2.4 GHz proyectan consumos dinámicos elevados bajo estrés computacional sostenido. Por ello, el análisis de diseño determinó que el suministro eléctrico debía desacoplarse totalmente entre nodos mediante fuentes dedicadas de 27 W con perfil Power Delivery, mitigando riesgos de subtensión transitoria (*undervoltage*) que pudieran degradar la frecuencia de reloj o reiniciar los nodos durante fases de cálculo intensivo.

En tercer lugar, se evaluaron los márgenes de disipación térmica. Dado que el estran-

gulamiento térmico (*thermal throttling*) en la Raspberry Pi 5 se activa automáticamente al alcanzar los 80°C reduciendo la frecuencia de la CPU, se estableció el criterio de que el sistema de enfriamiento activo por modulación de ancho de pulso (PWM) debía mantener la temperatura de los nodos con un margen de seguridad holgado respecto a dicho umbral crítico en todas las condiciones de reposo y operación.

Finalmente, se consideró analíticamente la asimetría de instrumentación entre plataformas: mientras que en el clúster ARM64 el sensor PMIC registra la potencia total disipada por la placa completa (procesador, memoria LPDDR4X y circuitería de soporte), la interfaz RAPL en x86 captura exclusivamente el dominio del encapsulado (`package-0`). Esta delimitación física fundamentó la necesidad analítica de formular dos métricas independientes de consumo (energía bruta y energía neta) para asegurar una evaluación comparativa transparente.

Mejorar. Con el fin de mitigar posibles degradaciones de rendimiento y asegurar la máxima disponibilidad operativa, se llevaron a cabo optimizaciones tanto en el sistema operativo como en la pila de software de alto rendimiento:

1. *Despliegue del gestor de colas Slurm:* Se instaló y configuró Slurm Workload Manager versión 23.11, desplegando el servicio central de control `slurmctld` en el nodo maestro y los servicios de ejecución `slurmd` en cada uno de los cinco nodos de cómputo, lo que permitió el despacho concurrente de tareas bioinformáticas.

2. *Aislamiento y control de recursos vía cgroups:* En entornos de memoria compartida (16 GB por nodo de cómputo), la ejecución concurrente de procesos científicos intensivos puede saturar la memoria física y activar el mecanismo *OOM Killer* (*Out-Of-Memory*) del kernel de Linux, congelando el nodo. Para prevenir esto, se integró el subsistema de grupos de control (`cgroups`) en el archivo de configuración `cgroup.conf` de Slurm, estableciendo techos máximos de memoria RAM por proceso (`AllowedRAMSpace=90 %`) y confinamiento estricto (`ConstrainRAMSpace=yes`), garantizando una reserva de memoria dedicada a las funciones críticas del sistema operativo y evitando desbordamientos de memoria.

3. *Almacenamiento distribuido compartido NFS:* Se implementó un servidor Network File System versión 4 (NFSv4) en el nodo maestro, exportando el directorio `/shared` hacia los cinco nodos de computo bajo las directivas de montaje `rw`, `sync`, `no_root_squash`, `no_subtree_check` con aceleración de lectura en búfer de memoria RAM local. Esta configuración unificó el acceso a los scripts, las bases de datos de referencia (GTDB y SILVA) y a los archivos FASTQ, evitando la redundancia de almacenamiento y eliminando la necesidad de duplicar decenas de gigabytes en las tarjetas MicroSD locales de cada trabajador.

4. *Configuración operativa del sistema operativo:* Cada nodo operó bajo Ubuntu Server 24.04 LTS en modalidad estrictamente sin interfaz gráfica (*headless*, objetivo `multi-user.target`), verificándose la ausencia de servidores de video activos y la inactividad de las interfaces inalámbricas (Bluetooth y Wi-Fi), asegurando que el flujo de datos y la telemetría operaran exclusivamente sobre el controlador Gigabit Ethernet cableado para evitar consumos parásitos de radiofrecuencia.

Controlar. Se establecieron protocolos automatizados de validación del estado y estabilidad del sistema. Mediante comandos nativos de Slurm (`sinfo`, `scontrol ping` y `squeue`) y scripts de prueba en Bash, se verificó la disponibilidad continua de los nodos de cómputo y la resolución bidireccional de red sin pérdidas de paquetes. Adicionalmente, se implementó un procedimiento de control de calidad para la telemetría energética: antes de autorizar las mediciones basales, se consultó el estado del PMIC en cada nodo mediante `vcgencmd pmic_read_adc`, validando la respuesta positiva y libre de anomalías en todos los canales de

sensado ($P > 0$) tras el período de estabilización de 60 segundos, asegurando la consistencia e integridad de las lecturas eléctricas.

2.4. Fase 2: Portabilidad de los pipelines a ARM64

En esta fase se abordó el reto de ingeniería de software consistente en portar, compilar y ejecutar de forma nativa en la arquitectura ARM64 (`aarch64`) tres pipelines bioinformáticos representativos para el análisis de amplicones 16S, asegurando su compatibilidad con el gestor de colas Slurm y optimizando su desempeño multihilo sobre el hardware del clúster.

Definir. Como marco de referencia se tomó LEMMI16s (Seppey y cols., 2025), una plataforma para evaluar y comparar herramientas de clasificación taxonómica de amplicones 16S. LEMMI16s incluye los pipelines empaquetados en contenedores Docker o Singularity, lo que facilita su instalación y despliegue, y genera muestras sintéticas a partir de una composición taxonómica conocida, de modo que la clasificación obtenida por cada pipeline se puede comparar con la composición real (*Ground Truth*). A partir de las herramientas disponibles en LEMMI16s se seleccionaron tres pipelines con enfoques algorítmicos distintos:

1. *Kraken 2 (v2.1.3) + Bracken (v3.0.1):* Enfoque basado en alineamiento libre de k-mers (*alignment-free*) y búsqueda exacta mediante minimizadores compactos indexados sobre la base de datos de referencia Genome Taxonomy Database (GTDB), seguido de una estimación de abundancias a nivel de especie mediante el algoritmo probabilístico de reestimación bayesiana de Bracken.

2. *LotuS3 Suite (v3.03):* Pipeline integral y ligero orientado al agrupamiento de secuencias en Unidades Taxonómicas Operacionales (OTUs). Integra el módulo de demultiplexación y filtrado de calidad estricto `sdm`, algoritmos de agrupamiento heurístico basados en identidad de secuencia (CD-HIT en el modo para lecturas largas PacBio) y asignación taxonómica mediante alineamiento con `vsearch` y el algoritmo del ancestro común más cercano (*Lowest Common Ancestor*, LCA) contra la base de datos SILVA 138.1.

3. *QIIME 2 (q2cli 2022.8):* Entorno modular de referencia en microbiología computacional que genera Variantes de Secuencia de Amplicones (ASVs) mediante denoising estadístico por modelado de tasas de error de secuenciación (algoritmo Deblur) y clasificación taxonómica automatizada basada en clasificadores Naive Bayes entrenados sobre bases de referencia (`q2-feature-classifier`).

Medir. Se realizó una auditoría de dependencias binarias y librerías dinámicas requeridas por cada pipeline. Se identificaron los módulos nativos en C, C++, Fortran y Perl,

así como la pila científica de Python (`numpy`, `scipy`, `scikit-learn`, `biopython`, `pandas`, `scikit-bio`, `h5py`). Para cada herramienta se evaluó la disponibilidad de paquetes precompilados para la arquitectura `aarch64` en canales comunitarios como Bioconda y Conda-Forge, identificando aquellas fuentes que exigían compilación manual desde el código fuente.

Analizar. El análisis técnico de la portabilidad sobre la plataforma Raspberry Pi 5 evidenció tres obstáculos principales:

1. *Restricción de contenedores Docker y Singularity/Apptainer:* Como se indicó al inicio de esta sección, LEMMI16s ejecuta sus pipelines dentro de contenedores Docker o Singularity/Apptainer. En el clúster, la construcción y ejecución de estos contenedores requería permisos de superusuario (`sudo`), los cuales no fueron otorgados a las cuentas del proyecto por la administración del clúster, dado que se trata de una infraestructura compartida en la que un cambio con privilegios de administrador podía afectar la configuración del sistema y el trabajo de otros usuarios. Adicionalmente, en Ubuntu Server 24.04 LTS el kernel aplica por omisión la directiva `kernel.apparmor_restrict_unprivileged_userns = 1`, que impide la creación de espacios de nombres de usuario no privilegiados, por lo que la invocación de contenedores sin privilegios fallaba con errores de permisos (`EPERM`).

2. *Inviabilidad de la emulación dinámica con QEMU:* Aunque es posible ejecutar binarios x86_64 en sistemas ARM64 mediante la capa de traducción QEMU User Emulation (`qemu-x86_64-static`), este mecanismo introduce una penalización considerable en el tiempo de ejecución y altera las mediciones de potencia al agregar la sobrecarga de traducir instrucciones. Dado que el objetivo del trabajo es medir la eficiencia del procesador ARM64 frente a x86, la emulación fue descartada.

3. *Incompatibilidad de herramientas y versiones entre plataformas:* No todas las herramientas y versiones disponibles para x86_64 contaban con soporte o paquetes precompilados para la arquitectura `aarch64`:

   En *QIIME 2*, no existían metapaquetes recientes de distribución para ARM64, siendo `q2cli 2022.8` la única versión funcional y estable en el clúster, mientras que la plataforma x86 contaba con distribuciones más actualizadas.

   En *Bracken*, el contenedor oficial de LEMMI16s en x86 empleaba la versión 2.8, pero los repositorios de Bioconda para `aarch64` únicamente disponían de la versión 3.0.1 (debido a la ausencia de compilaciones retrospectivas en la infraestructura ARM64 de Conda), lo que introducía divergencias en la estimación probabilística de abundancias.

En *LotuS3*, el alineador local predeterminado de la suite (`lambda3`) carece por completo de paquete o binario ejecutable para `aarch64`, forzando al clúster a recurrir a `vsearch`. Asimismo, componentes internos esenciales como el módulo de agrupamiento `usearch` y el clasificador `LCA` (*Lowest Common Ancestor*) se distribuían únicamente como binarios precompilados para x86_64, exigiendo compilación nativa desde código fuente en el clúster, mientras que ligeras discrepancias en la subversión de `vsearch` entre plataformas alteraban el número de lecturas asignadas en secuencias largas.

Estas discrepancias de software y dependencias arquitectónicas impedían establecer una comparación experimental válida y reproducible si no se realizaba una homologación estricta previa.

Mejorar. Para superar estos impedimentos se adoptaron las siguientes soluciones:

1. *Transición a ejecución nativa en espacio de usuario:* Se desacoplaron los pipelines de la capa de contenedores y se instalaron de forma nativa, sin requerir permisos de superusuario, mediante entornos de Conda para la arquitectura `aarch64` alojados en el almacenamiento compartido: `lemmi16s_env` (Kraken 2 + Bracken), `lemmi16s_pacbio_env` (LotuS3) y `qiime2_arm64` (QIIME 2). Kraken 2, Bracken y los componentes `sdm` y `vsearch` se instalaron como paquetes precompilados para ARM64 desde Bioconda. LotuS3 se instaló a partir del paquete de Bioconda descargado manualmente, ya que la resolución automática de dependencias fallaba en ARM64. El módulo de agrupamiento `usearch12` se compiló desde su código fuente oficial para `aarch64`, y QIIME 2 (q2cli 2022.8) se instaló mediante `pip` sobre un entorno Python 3.8 de Conda-Forge.

2. *Alineación de versiones entre plataformas:* Durante la fase exploratoria preliminar se identificaron discrepancias entre los paquetes disponibles en los repositorios de cada plataforma (por ejemplo, Bracken 2.8 en x86 frente a 3.0.1 en ARM64). Para asegurar que las comparaciones experimentales reflejaran las características de la microarquitectura y no discrepancias de software, para todas las corridas finales comparativas de rendimiento, telemetría y fidelidad taxonómica se homologaron estrictamente las versiones en ambas plataformas: Kraken 2.1.3, Bracken 3.0.1, LotuS 3.03 y QIIME 2 (q2cli 2022.8). En la plataforma x86, Bracken se reinstaló de forma nativa a partir del repositorio oficial para igualar la versión 3.0.1 del clúster, y el entorno de QIIME 2 se ajustó a q2cli 2022.8. En LotuS3 se fijó además el alineador de la asignación taxonómica: Dado que LAMBDA3 no tiene paquete para `aarch64`, el clúster cambia automáticamente a `vsearch`. Para que las tres plataformas ejecutaran el mismo flujo, LotuS3 se corrió en todos los equipos con `vsearch` v2.31.0 (opción `-taxAligner vsearch`). Todas las ta-

blas, figuras y resultados definitivos del presente trabajo corresponden exclusivamente a estas versiones homologadas.

3. *Orquestación distribuida vía scripts Slurm:* Se diseñaron scripts de sumisión por lotes que asignan cada muestra biológica de forma exclusiva a un nodo de cómputo (`-nodes=1 -exclusive`), operando sobre el almacenamiento compartido `/shared`. Para maximizar el aprovechamiento de los 4 núcleos Cortex-A76 sin saturar los buses de datos, Kraken 2 y LotuS3 se parametrizaron con 4 hilos por tarea, mientras que QIIME 2 se configuró con 2 hilos por tarea (`-cpus-per-task`) y un techo de contención de 14 GB (`-mem=14G`). Asimismo, como delimitación metodológica orientada a desacoplar el efecto de la aceleración distribuida frente a la eficiencia propia de la microarquitectura, se incorporó una condición experimental de control consistente en evaluar cada pipeline bajo ejecución secuencial (en un nodo físico de control) bajo las mismas restricciones operativas.

4. *Optimización de memoria y reducción del clasificador en QIIME 2:* El clasificador Naive Bayes (`q2-feature-classifier classify-sklearn`) entrenado sobre la base de datos taxonómica completa de SILVA (más de 500,000 secuencias no acotadas) demanda entre 32 y 64 GB de memoria RAM durante la clasificación, lo que provocaría el colapso inmediato por falta de memoria física (*Out-of-Memory*, OOM) tanto en los nodos de la Raspberry Pi 5 (con 16 GB de RAM física compartida) como en las computadoras portátiles. Para mitigar esta restricción, como adaptación metodológica, se implementó el procedimiento de reducción y acotamiento sobre la base SILVA 138.1, extrayendo un conjunto curado de aproximadamente 62,000 secuencias representativas focalizadas en las regiones hipervariables V1–V2 (`referenceClassificator.qza`, 118 MB). Esta optimización acotó la huella máxima estimada de memoria a menos de 3 GB por proceso, permitiendo una ejecución fluida con 2 hilos por nodo sin riesgo de estrangulamiento de memoria ni degradación por contención de recursos.

Controlar. Antes de iniciar las campañas de medición se verificó el funcionamiento de cada entorno nativo en el clúster y en las plataformas x86: se validó la homologación de versiones de cada herramienta (`kraken2 -version`, `bracken -v`, `lotus3 -v` y `qiime -version`) y se ejecutó una corrida de prueba piloto con una muestra biológica por pipeline para certificar la ausencia de fallos de segmentación. Adicionalmente, las secuencias FASTQ de entrada y el modelo clasificador entrenado (`referenceClassificator.qza`) fueron distribuidos de manera centralizada hacia el almacenamiento compartido NFS y las estaciones x86, validando su integridad criptográfica mediante sumas de verificación MD5 para asegurar que todas las plataformas procesaran exactamente las mismas entradas.

2.5. Fase 3: Validación de la integridad funcional

En esta fase se validó la integridad funcional y la fidelidad biológica de los pipelines bioinformáticos ejecutados sobre la arquitectura ARM64, comparando de manera sistemática sus salidas frente a los resultados obtenidos en las estaciones de control x86 y contrastándolos contra conjuntos de datos de referencia con verdad de terreno conocida (*Ground Truth*).

Definir. Para asegurar una evaluación imparcial, rigurosa y estandarizada, se establecieron tres conjuntos de datos sintéticos de amplicones 16S con una composición definida, generados por la plataforma de referencia internacional LEMMI16s (Seppey y cols., 2025), los cuales fueron:

1. *Dataset HOMD_v4_GTDB (Kraken 2 + Bracken):* Comunidad sintética representativa del microbioma oral humano generada a partir de secuencias del gen 16S rRNA de longitud completa, de las cuales se simularon lecturas pareadas de 200 pb con el perfil Illumina MiSeq v3. Comprende 2,453,431 lecturas pareadas distribuidas en 5 muestras independientes (`c001`, `c002`, `c003`, `e001` y `e002`), analizadas contra el índice taxonómico de GTDB.

2. *Dataset HM_Contaminated_Soil (LotuS3 Suite):* Comunidad simulada de suelo contaminado con metales pesados, con lecturas PacBio HiFi del gen 16S rRNA completo generadas por la versión de LEMMI16s para lecturas largas (herramienta MHASS, perfil PacBio Sequel). Comprende 25,611 lecturas de una sola hebra, con una longitud media de 1,505 pb, en 5 muestras: tres de calibración (`c001`, `c002` y `c003`), que se usan para elegir el umbral de filtrado que maximiza el F1, y dos de evaluación (`e001` y `e002`), con las que se mide el desempeño final. Las muestras se generaron una sola vez y se copiaron a las tres plataformas, verificando con sumas MD5 que fueran idénticas.

3. *Dataset alfa_v1v2_SILVA (QIIME 2):* Amplicones correspondientes a las regiones hipervariables V1–V2 del gen 16S rRNA secuenciados mediante tecnología Illumina, abarcando un total de 2,690,077 lecturas pareadas en 5 muestras biológicas (`c001`, `c002`, `c003`, `e001` y `e002`), evaluadas mediante el clasificador Naive Bayes entrenado sobre SILVA 138.

Medir. Se ejecutaron los tres pipelines utilizando idénticos archivos FASTQ de entrada, versiones de bases de datos y parámetros de filtrado tanto en la estación de trabajo x86 nativa como en el clúster RBPi5. Las salidas computacionales (tablas de características, perfiles de abundancia relativa y asignaciones taxonómicas) se transformaron a matrices tabulares unificadas. Se definieron formalmente las métricas estadísticas para cuantificar la fidelidad

biológica de cada pipeline frente al archivo oficial del Ground Truth (`queryTaxo.tsv`):

- *Verdaderos Positivos (TP):* Número de taxones (familias, géneros o especies) identificados por el pipeline que coinciden biológica y taxonómicamente con los taxones presentes en el *Ground Truth*.
- *Falsos Positivos (FP):* Número de taxones predichos por el pipeline que no están presentes en la verdad de terreno (asignaciones espurias o ruido metagenómico).
- *Falsos Negativos (FN):* Número de taxones presentes en el *Ground Truth* que el pipeline no logró detectar (omisiones taxonómicas).
- *Taxones de Verdad de Terreno (GT):* Número total de linajes biológicos verdaderos contenidos en la muestra simulada ($GT = TP + FN$).

A partir de estas variables se calcularon la Precisión (P), la Sensibilidad (*Recall*, R) y la media armónica o puntuación F1 (*F1-Score*, F_1) para cada muestra y nivel taxonómico:

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = 2 \cdot \frac{P \cdot R}{P + R} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \tag{2.1}$$

Adicionalmente, para verificar la reproducibilidad cruzada entre plataformas (x86 frente a ARM64), se calculó la concordancia taxonómica absoluta mediante la comparación diferencial de archivos línea por línea (`diff -u`), la verificación de coincidencia en los linajes taxonómicos detectados y la comparación de los perfiles de abundancia relativa.

Analizar. El análisis de la integridad biológica se estructuró a partir de dos niveles complementarios de evaluación entre las salidas generadas en la arquitectura ARM64 y las estaciones de control x86:

En primer lugar, se estableció un análisis computacional orientado a verificar el determinismo algorítmico entre plataformas mediante la inspección diferencial directa (`diff -u`) de los archivos de predicción taxonómica y perfiles de abundancia. Este análisis permitió evaluar si las diferencias arquitectónicas en los conjuntos de instrucciones o en las unidades de coma flotante introducían variaciones en los métodos deterministas (como la concordancia de k-mers en Kraken 2 o el modelado de Deblur en QIIME 2).

En segundo lugar, se formuló un análisis de consistencia biológica enfocado en comparar la robustez taxonómica frente a la verdad de terreno. Dado que flujos como LotuS3 incorporan etapas de agrupamiento heurístico y alineamiento local en secuencias de longitud completa (PacBio HiFi), el análisis se centró en corroborar si posibles variaciones menores en la agregación estocástica de lecturas alteraban la identificación de linajes clave o si, por el

contrario, los perfiles taxonómicos a nivel de familia y género se mantenían biológicamente equivalentes en todas las plataformas.

Con base en estos dos ejes, se definieron los criterios de aceptación analítica: para validar la reproducibilidad entre arquitecturas se requirió la preservación de los taxones biológicos verdaderos y la invariabilidad de las métricas de clasificación (P, R y F_1), asegurando que cualquier cambio de plataforma no introdujera sesgos metodológicos ni divergencias en la interpretación biológica de las comunidades microbianas.

Mejorar. Se ajustaron los parámetros de umbral de filtrado de calidad y longitud mínima de lecturas en los scripts de ejecución para garantizar que ningún pipeline descartara fragmentos válidos de secuencias por diferencias en la interpretación de caracteres ASCII. Asimismo, se unificaron las versiones de los índices de referencia en el volumen NFS para evitar discrepancias temporales en la resolución de linajes taxonómicos.

Controlar. Las tablas de taxonomía resultantes de cada corrida se evaluaron con el procedimiento de análisis del benchmark LEMMI16s, que compara las predicciones frente al archivo de referencia `queryTaxo.tsv` para obtener las métricas de la Ecuación (2.1). Al revisar ese procedimiento se encontró que el evaluador de LEMMI16s calcula todas las muestras de evaluación con la verdad de terreno de la primera de ellas (`input.truth[0]` en el archivo `eval.smk`), por lo que el F1 se recalculó con la misma lógica de LEMMI16s (umbral de calibración y normalización por número de copias del gen), pero usando para cada muestra su propio archivo `queryTaxo.tsv`. Adicionalmente, las predicciones del clúster se compararon directamente con las obtenidas en x86 mediante `diff -u`, verificando que las diferencias entre plataformas fueran nulas o se limitaran a variaciones menores en los conteos.

2.6. Fase 4: Evaluación de desempeño y eficiencia energética

En esta fase se evaluó experimentalmente el desempeño computacional, el consumo eléctrico y la eficiencia energética de las tres plataformas de cómputo bajo estudio, estableciendo un protocolo estandarizado de medición para formular la relación de eficiencia en lecturas procesadas por vatio-hora (*Reads/Wh*).

Definir. Se definió un protocolo de medición común para las tres plataformas, con el fin de que cada corrida comenzara con el equipo sin carga y sin actividad previa que alterara la potencia medida. Cada sesión experimental se estructuró en tres fases continuas:

1. *Estabilización (60 segundos, sin medición):* Tras lanzar el script de medición, el sistema permanece sin carga para que terminen los procesos residuales del arranque y la actividad previa.

2. *Línea base previa en reposo (Pre-run idle, 5 minutos):* El sistema se mantiene sin carga de procesamiento y se registra la potencia a 1.0 Hz. Su promedio es la potencia basal $\bar{P}_{\text{idle}}$ de esa sesión.

3. *Fase de cómputo activo (Active computation):* Se ejecuta el pipeline bioinformático completo bajo orquestación de Slurm en el clúster o mediante ejecución nativa en las estaciones x86, registrando de manera continua a 1.0 Hz la potencia instantánea disipada $P(\tau)$.

La estandarización de este protocolo responde a la necesidad de controlar la deriva térmica residual: ensayos preliminares de calibración indicaron que intervalos de reposo breves (inferiores a un minuto) son susceptibles a sobrestimaciones de la potencia basal debido al calor acumulado y a procesos residuales del sistema operativo inmediatamente posteriores a una tarea previa. Por esta razón, se adoptó formalmente un intervalo de estabilización pasiva de 60 segundos seguido de 300 segundos (5 minutos) de registro continuo, garantizando una estimación robusta y asintóticamente estable de $\bar{P}_{\text{idle}}$ para cada sesión experimental.

Además, para separar el efecto del paralelismo, cada pipeline se evaluó bajo un esquema de ejecución secuencial en el clúster (QIIME 2 y LotuS3 en `rbp5-1`, `rbp5-2`, y Kraken 2 en `rbp5-3`), procesando todas sus muestras una tras otra con el mismo protocolo, los mismos hilos por tarea y las mismas entradas que en la ejecución distribuida, lanzado mediante Slurm con acceso exclusivo al nodo.

Medir. A lo largo de la ejecución de cada pipeline se adquirieron concurrentemente las siguientes variables físicas y computacionales:

- *Tiempo de ejecución (t):* Duración en segundos de la ventana de medición de la fase activa, desde la primera hasta la última lectura de potencia. Es el mismo intervalo sobre el que se integra la energía. En el clúster corresponde a la ventana conjunta de los nodos, desde el primero que inicia hasta el último que termina.

- *Carga de CPU (%):* Porcentaje promedio de utilización de los núcleos de procesamiento muestreado a través del subsistema `procfs` de Linux (`/proc/stat`) y reportado a nivel de sistema.

- *Memoria RAM máxima (GB):* Huella máxima de memoria física residente (*Resident Set Size*, RSS) ocupada por los procesos bioinformáticos a lo largo del flujo.

- *Potencia media en carga ($\bar{P}_{activa}$):* Promedio temporal de la potencia eléctrica total disipada durante la fase activa, expresada en vatios (W).
- *Potencia media en reposo ($\bar{P}_{idle}$):* Potencia eléctrica basal registrada durante el intervalo previo de reposo, correspondiente al costo energético de mantener la plataforma encendida y operativa.

Analizar. Para analizar de forma física y matemáticamente rigurosa el gasto energético, se formuló el balance eléctrico del sistema. La potencia neta atribuible exclusivamente al cálculo computacional (P_{neta}) se calculó sustrayendo el consumo basal en reposo de la potencia activa:

$$P_{\text{neta}}(\tau) = P_{\text{activa}}(\tau) - \bar{P}_{\text{idle}} \qquad [\text{W}] \tag{2.2}$$

A partir de la potencia instantánea se calculó la energía total consumida (E_{total}) y la energía neta computacional (E_{neta}), integradas en el dominio temporal y expresadas en vatios-hora (Wh). Como la potencia se registra en lecturas discretas P_i tomadas en los instantes t_i ($i = 1, \ldots, n$), la integral se evaluó numéricamente mediante la regla del trapecio sobre las marcas de tiempo reales de cada lectura:

$$E_{\text{total}} = \frac{1}{3600} \int_{t_1}^{t_n} P_{\text{activa}}(\tau)\, d\tau \approx \frac{1}{3600} \sum_{i=1}^{n-1} \frac{P_i + P_{i+1}}{2} \left(t_{i+1} - t_i\right) \qquad [\text{Wh}] \tag{2.3}$$

$$E_{\text{neta}} = \frac{1}{3600} \int_{t_1}^{t_n} P_{\text{neta}}(\tau)\, d\tau \approx \frac{1}{3600} \sum_{i=1}^{n-1} \left(\frac{P_i + P_{i+1}}{2} - \bar{P}_{\text{idle}} \right) \left(t_{i+1} - t_i\right) \qquad [\text{Wh}] \tag{2.4}$$

La evaluación numérica mediante la regla del trapecio sobre marcas de tiempo reales ($t_{i+1} - t_i$), en lugar de asumir un paso de discretización constante, se fundamenta en el comportamiento temporal del sistema operativo: bajo cargas computacionales intensivas, la contención en la planificación de procesos en espacio de usuario puede introducir pequeñas fluctuaciones en la frecuencia efectiva de muestreo de la telemetría. La integración temporal sobre intervalos reales garantiza la exactitud del cómputo energético acumulado. En el clúster, la energía se calculó de forma independiente para cada nodo de cómputo empleando su respectivo $\bar{P}_{\text{idle}}$, computando posteriormente el consumo agregado del sistema.

La energía total (E_{total}) es toda la energía que consumió el equipo durante la corrida, incluida la que gasta solo por estar encendido. La energía neta (E_{neta}) es la energía total menos la que el equipo habría consumido en reposo durante el mismo tiempo ($\bar{P}_{\text{idle}} \cdot t$), es decir, la energía adicional que exigió el pipeline. La energía total refleja lo que realmente se paga en electricidad, mientras que la energía neta permite comparar cuánto consume el cómputo en sí, sin el efecto del consumo en reposo de cada plataforma.

Para cuantificar el rendimiento biológico por unidad de energía eléctrica consumida, se formuló la métrica principal de eficiencia energética (η), definida como el número total de lecturas biológicas procesadas (N_{reads}) por cada vatio-hora invertido:

$$\eta_{\text{bruta}} = \frac{N_{\text{reads}}}{E_{\text{total}}} \quad \left[\frac{\text{reads}}{\text{Wh}}\right], \qquad \eta_{\text{neta}} = \frac{N_{\text{reads}}}{E_{\text{neta}}} \quad \left[\frac{\text{reads}}{\text{Wh}}\right] \tag{2.5}$$

Finalmente, para cuantificar la ventaja relativa de una plataforma sobre otra, se definió el factor de ganancia verde o aceleración energética relativa (G_{verde}) como la razón entre la eficiencia alcanzada por el clúster ARM64 frente a la estación x86:

$$G_{\text{verde}} = \frac{\eta_{\text{clúster}}}{\eta_{\text{x86}}} \tag{2.6}$$

Mejorar. Con el fin de aislar estrictamente el costo energético del procesamiento biológico y evitar sobreestimaciones por etapas accesorias, se estableció como directriz metodológica la preindexación de las bases de datos de referencia: tanto el índice de `vsearch` para LotuS3 (archivo en formato UDB de aproximadamente 3 GB) como los índices taxonómicos de Kraken 2 se generaron de forma centralizada previo a las sesiones de prueba. De este modo, la ventana de telemetría capturó de manera pura el consumo derivado del control de calidad, agrupamiento, inferencia y clasificación taxonómica de las muestras, eliminando el sesgo de las fases de preparación de datos.

Controlar. Para asegurar la validez y comparabilidad de las mediciones, cada corrida experimental se normalizó exclusivamente frente al reposo basal ($\bar{P}_{\text{idle}}$) registrado en su propia sesión y entorno térmico. Asimismo, se estableció un protocolo de verificación de estabilidad que descartó cualquier sesión experimental que presentara derivas basales o anomalías en la telemetría al finalizar el procesamiento, garantizando que las integrales de energía neta calculadas reflejaran de forma fidedigna el esfuerzo computacional de los pipelines bioinformáticos.

Capítulo 3

Resultados

En este capítulo se presentan los resultados obtenidos en cada una de las fases del proyecto. En todos los casos, la comparación principal se realiza entre el clúster RBPi5 y el computador x86 con Ubuntu nativo. Los resultados obtenidos en WSL2 se incluyen como referencia complementaria.

3.1. Fase 1: Clúster RBPi5 y línea base energética

El clúster quedó conformado como se describió en la Sección 2.3 y se muestra en la Figura 3.2: un nodo maestro Raspberry Pi 4B, que solo coordina las tareas mediante Slurm y comparte el directorio `/shared` por NFS, y diez nodos de cómputo Raspberry Pi 5 conectados por un switch Gigabit Ethernet, de los cuales cinco fueron asignados al proyecto (`rbp5-1` a `rbp5-5`). Antes de cada corrida se verificó con `sinfo` que los cinco nodos de cómputo estuvieran disponibles en estado `idle` (Figura 3.1). La Tabla 3.1 presenta la potencia en reposo de cada nodo de cómputo, medida con el PMIC de la placa durante los 5 minutos previos a cada corrida.

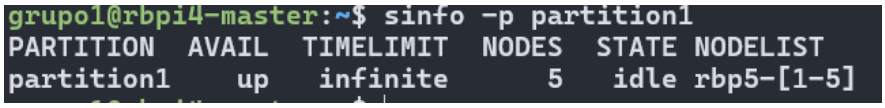


Figura 3.1: Estado de los cinco nodos de cómputo en la partición `partition1` de Slurm. Fuente: Propia.



Figura 3.2: Clúster RBPi5 en operación: nodo maestro Raspberry Pi 4B (caja roja), switch Gigabit Ethernet y dos torres de cinco nodos de cómputo Raspberry Pi 5. Fuente: Propia.

Tabla 3.1: Potencia en reposo (*stand-by*) de los nodos de cómputo del clúster RBPi5, promedio de las mediciones de 5 minutos previas a cada corrida.

Nodo de cómputo	**Potencia en reposo (W)**
`rbp5-1`	2.389
`rbp5-2`	2.377
`rbp5-3`	2.420
`rbp5-4`	2.379
`rbp5-5`	2.332
Total (5 nodos de cómputo)	**11.896**

El nodo maestro (Raspberry Pi 4B) no se incluye en esta caracterización, ya que no incorpora el PMIC DA9091 y no participa en la ejecución de los pipelines. Su función se limita a la coordinación de tareas mediante Slurm y al servicio de almacenamiento compartido NFS.

3.2. Fase 2: Pipelines portados a ARM64

Se lograron portar, compilar y ejecutar con éxito los tres pipelines de análisis de amplicones 16S sobre la arquitectura ARM64 con Ubuntu Server 24.04 LTS. La Tabla 3.2 detalla las herramientas adaptadas, sus versiones y los ajustes técnicos requeridos.

Tabla 3.2: Pipelines portados a la arquitectura ARM64.

Pipeline	**Versión**	**Método de instalación**	**Ajustes necesarios en ARM64**
Kraken 2 + Bracken	Kraken 2.1.3 Bracken 3.0.1	Entorno Conda nativo (`lemmi16s_env`)	Paquetes precompilados aarch64 de Bioconda. Mapeo de memoria para base de datos GTDB (142 MB).
LotuS3	LotuS 3.03	Entorno Conda + binarios nativos ARM64	Paquete de Bioconda instalado manualmente (`-no-deps`), `sdm`, `vsearch` de Bioconda y `usearch12` compilado desde código fuente. Asignación taxonómica con `vsearch`, porque LAMBDA3 no está disponible para ARM64. Indexación previa de la base de referencia en formato UDB.
QIIME 2	2022.8 (Core)	Entorno Conda dedicado (`qiime2_arm64`, Python 3.8)	Instalación de q2cli 2022.8 y Deblur 1.1.0 mediante `pip` sobre Python 3.8. Reducción de la base SILVA 138 a regiones V1–V2 ($\approx$ 62,000 secuencias, 118 MB) para mitigar colapso por memoria (OOM). Asignación de 2 hilos por nodo.

Los obstáculos encontrados durante la portabilidad y las soluciones adoptadas se describieron en la Sección 2.4.

La Figura 3.3 muestra la ejecución, a través de Slurm, de la verificación de versiones en el nodo de cómputo `rbp5-1`: el sistema reporta la arquitectura `aarch64` y las versiones instaladas de los tres pipelines, que son las mismas usadas en las plataformas x86 y en todos los resultados de este capítulo.

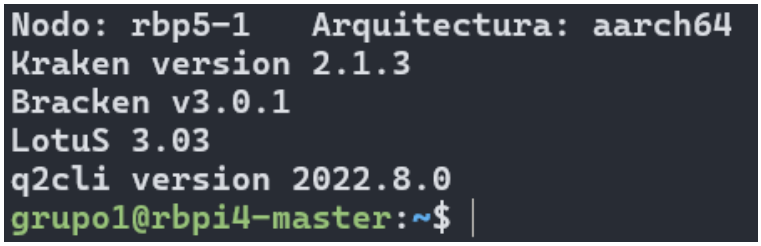


Figura 3.3: Arquitectura y versiones de los pipelines en el nodo de cómputo `rbp5-1` del clúster. Fuente: Propia.

3.3. Fase 3: Validación de resultados biológicos

Los tres conjuntos de datos de LEMMI16s utilizados y sus características se describieron en la Sección 2.5.

La Tabla 3.3 contrasta las métricas biológicas y taxonómicas obtenidas en las tres plataformas evaluadas frente a la verdad de terreno (*Ground Truth*) oficial (`queryTaxo.tsv`).

Tabla 3.3: Comparación de resultados biológicos y taxonómicos frente al *Ground Truth*.

Pipeline	Métrica de Evaluación	x86 (Ubuntu nativo)	x86 (WSL2)	Clúster RBPi5
Kraken 2 + Bracken	Lecturas biológicas procesadas	2,453,431	2,453,431	2,453,431
	Taxones predichos por muestra (Fam. / Gén.)	159–181 / 377–457	159–181 / 377–457	159–181 / 377–457
	F1-Score promedio (Familia / Género)	0.5223 / 0.3545	0.5223 / 0.3545	0.5223 / 0.3545
	Concordancia frente a Nativo (*diff -u*)	*Referencia base*	100 % identidad bit a bit	100 % identidad bit a bit
LotuS3 Suite	Lecturas biológicas procesadas (PacBio HiFi)	25,611	25,611	25,611
	Taxones predichos por muestra (Fam. / Gén.)	16–18 / 28–34	16–18 / 28–34	16–18 / 28–34
	F1-Score promedio (Familia / Género)	0.8667 / 0.8762	0.8667 / 0.8762	0.8667 / 0.8762
	Concordancia taxonómica frente a Nativo	*Referencia base*	Mismos géneros, F1 idéntico	Mismos géneros, F1 idéntico
QIIME 2	Lecturas biológicas procesadas	2,690,077	2,690,077	2,690,077
	Rango de ASVs colapsados (L7)	8–10 Fam. / 8–11 Gén.	8–10 Fam. / 8–11 Gén.	8–10 Fam. / 8–11 Gén.
	F1-Score promedio (Fam. / Gén. / Esp.)	0.8286 / 0.7083 / 0.4500	0.8286 / 0.7083 / 0.4500	0.8286 / 0.7083 / 0.4500
	Concordancia frente a Nativo (*diff -u*)	*Referencia base*	100 % identidad bit a bit	100 % identidad bit a bit

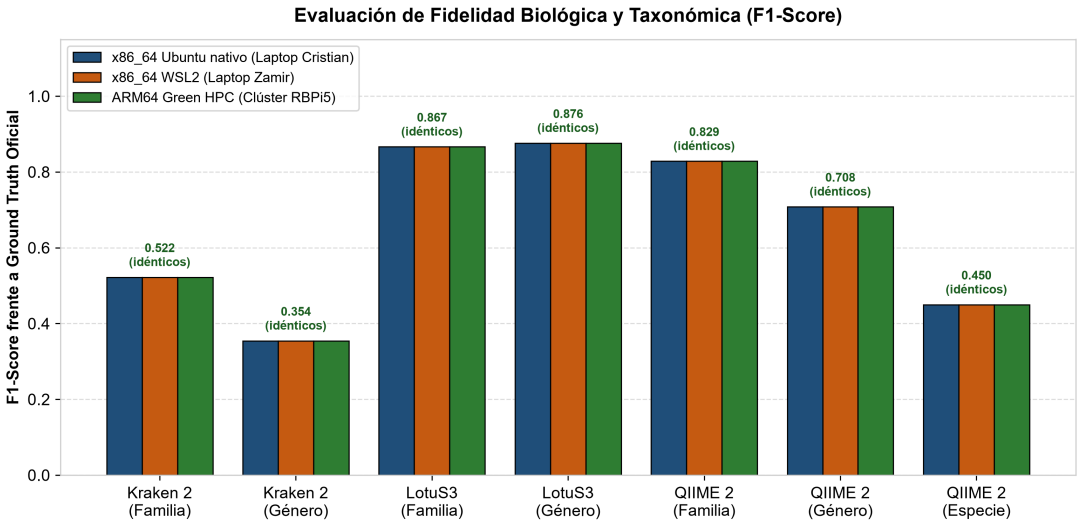


Figura 3.4: Fidelidad taxonómica (F1-Score) frente al *Ground Truth*: comparación de reproducibilidad biológica entre las estaciones x86_64 (Ubuntu nativo y WSL2) y el clúster ARM64.

Como se observa en la Figura 3.4, los resultados taxonómicos del clúster ARM64 coincidieron con los obtenidos en x86 con Ubuntu nativo en los tres pipelines, y el F1 fue el mismo en las tres plataformas. Los valores de F1 se calcularon con la verdad de terreno de cada muestra, como se explicó en la Sección 2.5.

En QIIME 2, las tablas de asignación taxonómica fueron idénticas bit a bit en las tres plataformas en las cinco muestras, con un F1 promedio de 0.8286 en familia, 0.7083 en género y 0.4500 en especie, y una precisión media de 0.89 en familia y género.

En Kraken 2 + Bracken, las predicciones también fueron idénticas bit a bit en las tres plataformas en las cinco muestras: entre 159 y 181 familias y entre 377 y 457 géneros por muestra, con un F1 promedio de 0.5223 en familia y 0.3545 en género. Este F1 moderado refleja la dificultad de clasificar una comunidad bacteriana con muchos géneros cercanos, y no depende de la plataforma, ya que el resultado fue el mismo en las tres.

Por último, en LotuS3 las tres plataformas obtuvieron el mismo F1, 0.8667 en familia y 0.8762 en género, con entre 16 y 18 familias y entre 28 y 34 géneros por muestra. Las dos laptops x86 asignaron prácticamente las mismas lecturas a cada género (entre 0 y 4 lecturas de diferencia por muestra) y, frente al clúster, la diferencia fue de 35 a 201 lecturas por muestra, sin cambiar los géneros detectados. El F1 de LotuS3 es el más alto de los tres pipelines en género, lo que se explica porque cada lectura cubre el gen 16S completo (unos 1,500 pb), y con esa longitud es más fácil distinguir entre géneros cercanos que con fragmentos cortos de una región hipervariable.

3.4. Fase 4: Desempeño computacional y eficiencia energética

La evaluación del desempeño computacional y el comportamiento energético se fundamentó en un protocolo experimental estandarizado con telemetría continua a 1.0 Hz. Cada ensayo inició con 60 segundos de estabilización sin medición en reposo absoluto sin captura, seguidos de 300 segundos (5 minutos) continuos para caracterizar la potencia de línea base (P_{idle}), y un registro ininterrumpido durante toda la fase de procesamiento activo. Más allá de contrastar cifras brutas, el análisis se articula en torno a la relación de compromiso (*trade-off*) entre el costo en tiempo de la microarquitectura ARM64 y su retorno en términos de eficiencia energética y paralelismo distribuido.

En esta sección se presentan, en primer lugar, los resultados comparativos globales entre la ejecución secuencial y distribuida en el clúster frente a las plataformas x86. Seguidamente, se analiza de forma individual el comportamiento de cada pipeline bioinformático, evaluando el tiempo de ejecución, la demanda de recursos de cómputo y los perfiles dinámicos de potencia eléctrica.

3.4.1. Ejecución secuencial frente a distribuida

Como primer experimento, cada pipeline se evaluó bajo un esquema de ejecución secuencial en el clúster, procesando las 5 muestras una tras otra con el mismo protocolo de medición, los mismos hilos por tarea y las mismas entradas que en las demás plataformas. El objetivo es comparar el desempeño de la arquitectura ARM64 con las dos laptops x86, que también procesan las muestras de manera secuencial. Posteriormente, se evaluó la ejecución distribuida, asignando concurrentemente cada muestra a un nodo de cómputo del clúster. La Tabla 3.4 y las Figuras 3.5 y 3.6 resumen los resultados.

Tabla 3.4: Tiempo y energía acumulada por plataforma: laptops x86 y clúster bajo ejecución secuencial y distribuida.

Pipeline	Métrica	x86 (Ubuntu nativo)	x86 (WSL2)	Clúster 1 nodo	Clúster 5 nodos (nodo por muestra)
Kraken 2 + Bracken	Tiempo	22.01 s	51.41 s	98.35 s	51.04 s (5 nodos)
	Energía total (Wh)*	0.2183	0.3744	0.0976	0.2023
	Energía neta (Wh)*	0.1518	0.1775	0.0321	0.0398
LotuS3	Tiempo	31.65 min	29.60 min	143.0 min	32.9 min (5 nodos)
	Energía total (Wh)*	17.1442	17.6435	14.2991	12.0311
	Energía neta (Wh)*	15.6519	10.1478	8.6506	7.3270
QIIME 2	Tiempo	72.06 min	79.99 min	139.9 min	28.0 min (5 nodos)
	Energía total (Wh)*	25.4939	34.2407	8.4246	8.2884
	Energía neta (Wh)*	12.6965	16.9772	2.9144	2.7779

* Energía acumulada durante la corrida completa (no promedio).

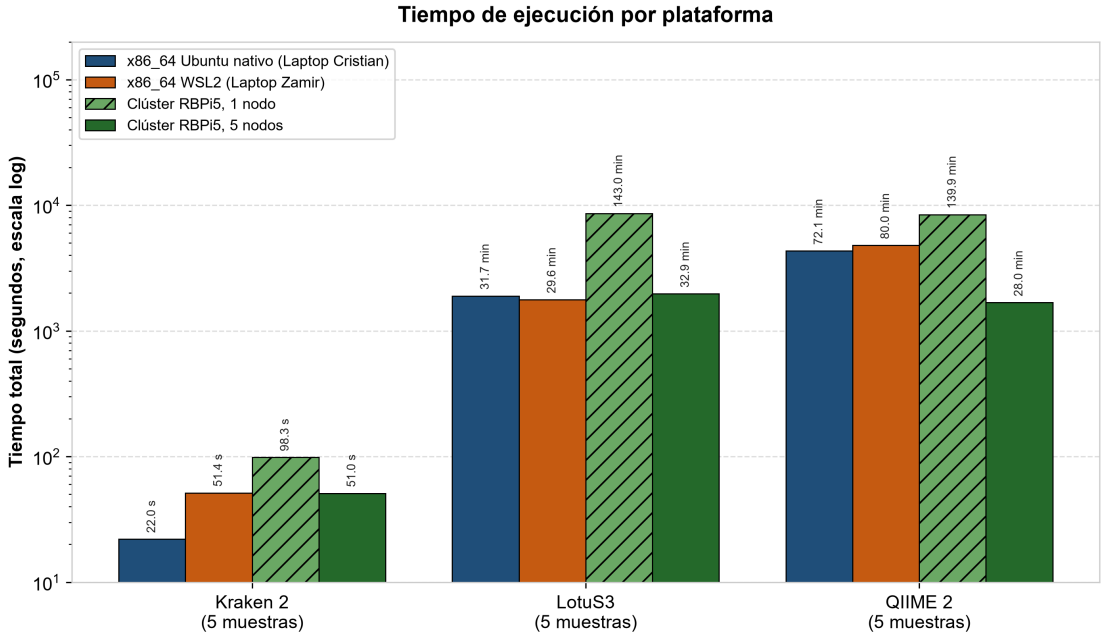


Figura 3.5: Tiempo de ejecución por plataforma, evaluando la ejecución secuencial y distribuida sobre 5 nodos.

Como se observa en la Figura 3.5, bajo la ejecución secuencial el clúster completó los análisis con tiempos superiores a la estación x86 nativa: 4.47× más lento en Kraken 2 (98.35 frente a 22.01 s), 4.52× en LotuS3 (143.0 frente a 31.65 min) y 1.94× en QIIME 2 (139.9 frente a 72.06 min). Esta brecha responde a la disparidad de potencia de cálculo monohilo entre los núcleos ARM Cortex-A76 de la microarquitectura embebida y los núcleos del procesador Intel Core i5. No obstante, al transicionar hacia el esquema de ejecución distribuida (barra verde oscura), la asignación concurrente de muestras en los cinco nodos mitiga esta limitación

y transforma el comportamiento temporal de cada flujo:

En **QIIME 2**, el clúster exhibió un escalamiento casi lineal de 5.00× frente a su versión secuencial (reduciendo el tiempo global de 139.9 a 27.98 min). Debido a que este pipeline está dominado por etapas algorítmicas de inferencia y opera sobre un modelo taxonómico acotado (118 MB) que no congestiona el bus de datos, los cinco nodos procesaron sus respectivas muestras en paralelo sin experimentar cuellos de botella en la red NFS. Esta aceleración permitió que el clúster distribuido superara de manera contundente a las plataformas x86, completando el lote 2.58× más rápido que la estación x86 nativa (72.06 min) y 2.86× más rápido que WSL2 (79.99 min).

En **LotuS3**, el beneficio del paralelismo distribuido fue igualmente sustancial (4.35× de aceleración interna, pasando de 143.0 a 32.87 min), alcanzando un tiempo prácticamente equivalente al de la laptop nativa (0.96×, 32.9 frente a 31.7 min) y cercano a WSL2 (29.6 min). Sin embargo, la necesidad de transferir simultáneamente la base de referencia de `vsearch` (3 GB) a través del enlace Gigabit introdujo variaciones en el tiempo de procesamiento de cada nodo (entre 14 y 33 min), provocando que la duración del lote quedara fijada por el nodo más rezagado.

Por el contrario, **Kraken 2 + Bracken** evidenció el impacto crítico de la contención en red en flujos ultracortos: la aceleración distribuida fue de apenas 1.93× (de 98.35 a 51.04 s). Mientras que en la ejecución secuencial cada muestra tardó entre 17 y 21 s, en el esquema distribuido los cinco nodos compitieron concurrentemente en el instante inicial por transferir la base GTDB desde el nodo maestro NFS, elevando la latencia de carga a 50 s por nodo. Por este motivo, la estación x86 nativa, respaldada por almacenamiento SSD NVMe local de alta velocidad y memoria caché directa, mantuvo una ventaja de 2.32× en velocidad frente al clúster (22.01 s).

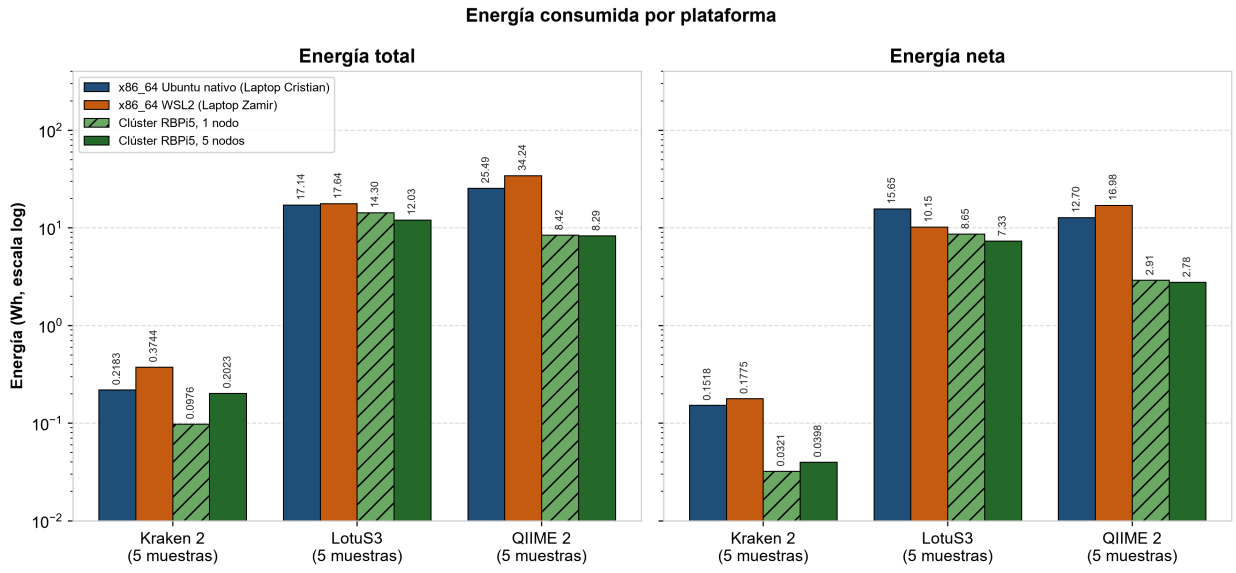


Figura 3.6: Energía total y neta por plataforma, evaluando la ejecución secuencial y distribuida en el clúster.

Como se muestra en la Figura 3.6, en el caso del desempeño energético, las barras del clúster se sitúan sistemáticamente por debajo de las estaciones x86 en los tres pipelines evaluados, tanto en la ejecución secuencial como en la distribuida, alcanzando reducciones notables en Kraken 2 y QIIME 2. En energía neta (gráfica derecha), la ejecución secuencial en el clúster consumió 4.7× menos energía que la laptop nativa en Kraken 2 (0.0321 frente a 0.1518 Wh), 1.8× menos en LotuS3 (8.65 frente a 15.65 Wh) y 4.4× menos en QIIME 2 (2.91 frente a 12.70 Wh). Frente a WSL2, la reducción fue de 5.5× en Kraken 2, 1.2× en LotuS3 y 5.8× en QIIME 2. Esta ventaja se sustenta en que una Raspberry Pi 5 opera con una potencia activa de tan solo 3 a 4 W en Kraken 2 y QIIME 2, y cerca de 6 W en LotuS3, frente al rango de 17 a 36 W demandado por la laptop. En energía total (gráfica izquierda), que incorpora el consumo acumulado en reposo, la ventaja del clúster en modo secuencial frente a la laptop nativa fue de 2.2× en Kraken 2, 1.2× en LotuS3 y 3.0× en QIIME 2.

Cabe resaltar que, para el caso de QIIME 2 y LotuS3, el consumo energético total fue prácticamente el mismo entre la ejecución secuencial y la distribuida (8.42 frente a 8.29 Wh en QIIME 2, y 14.30 frente a 12.03 Wh en LotuS3). Esto demuestra que repartir las muestras en múltiples nodos no incrementa el gasto eléctrico del sistema, debido a que la carga total de cómputo requerida por el análisis biológico se preserva sin penalizaciones energéticas por concurrencia. Incluso, en LotuS3 el esquema distribuido consumió ligeramente menos energía total que el secuencial, ya que cada nodo retornó a su estado basal de reposo (2.38 W) inmediatamente después de procesar su muestra.

En Kraken 2, por el contrario, la energía total bajo ejecución distribuida fue el doble que en la secuencial (0.2023 frente a 0.0976 Wh). Este incremento no obedeció a la demanda computacional del procesador, sino a la penalización temporal impuesta por el almacenamiento de red: mantener las cinco placas encendidas durante la contención de I/O por la descarga concurrente de la base de datos acumuló mayor energía en reposo que haber procesado las muestras una tras otra sobre un solo nodo.

Finalmente, se puede observar que, en conjunto, el sustancial ahorro energético reportado proviene de la alta eficiencia por vatio de la microarquitectura ARM64, mientras que la reducción en los tiempos de cómputo depende de la capacidad de paralelismo distribuido. Asimismo, las tablas taxonómicas obtenidas bajo ambos esquemas fueron idénticas en QIIME 2 y Kraken 2, y mantuvieron las mismas familias, géneros y puntuación F1 en LotuS3, certificando que el paso al esquema distribuido no induce ninguna divergencia biológica.

3.4.2. Análisis de desempeño y perfiles de potencia: Kraken 2 + Bracken

La Tabla 3.5 detalla el comportamiento de este flujo al procesar 2,453,431 lecturas biológicas. Solo se presenta el análisis de la versión distribuida ya que como se presentó en la sección anterior, esta supera el tiempo de ejecución de la versión secuencial sin introducir mayores penalidades en el desempeño energético.

Tabla 3.5: Métricas de desempeño computacional y energético: Kraken 2 + Bracken (5 muestras, 2,453,431 lecturas, protocolo 5 min de reposo basal).

Métrica de Evaluación	**x86 (Ubuntu nativo)**	**x86 (WSL2)**	**Clúster RBPi5**
Tiempo de ejecución (ventana medida, s)	22.01	51.41	51.04 (paralelo 5 nodos)
Uso medio de CPU (%)	94.20	88.50	91.50 (por nodo)
RAM máxima por proceso (GB)	1.80	1.80	1.60 (por nodo)
Potencia en reposo (P_{idle} 5 min) (W)	10.88	13.78	2.35 (por nodo)
Potencia media en carga (W)	35.70	26.22	2.93 (por nodo)
Potencia neta de cómputo (W)	24.83	12.43	0.58 (por nodo)
Energía total consumida (Wh)*	0.2183	0.3744	0.2023
Energía neta consumida (Wh)*	0.1518	0.1775	0.0398

* Energía acumulada durante la corrida completa (no promedio). En el clúster RBPi5 representa el consumo consolidado de los cinco nodos de cómputo.

En Kraken 2 la estación x86 nativa fue la más rápida: completó el lote en 22.01 s, frente a 51.04 s del clúster y 51.41 s de WSL2. La clasificación depende de tener la base de datos en memoria. En la laptop nativa la base se lee del disco local y queda en caché después de la primera muestra, mientras que en el clúster los cinco nodos la cargan al mismo tiempo desde el almacenamiento compartido NFS. Por eso, en la ejecución secuencial cada muestra

tardó entre 17 y 21 s, pero en la ejecución distribuida cada nodo tardó unos 50 s. Aun así, el clúster consumió menos energía: sus cinco nodos sumaron unos 14.6 W en carga, frente a 35.70 W de la laptop nativa, y su energía neta fue de 0.0398 Wh frente a 0.1518 Wh.

Un análisis detallado mostró que en el clúster, la mayor parte de la energía total (0.1625 de 0.2023 Wh) corresponde al reposo de los cinco nodos, porque la corrida dura más y la carga sobre cada nodo es baja. Por eso la ventaja del clúster es grande en energía neta (3.81×) y pequeña en energía total (1.08×).

Como se observa en la Figura 3.7, los cinco nodos se comportan de forma casi idéntica. En los primeros 5 segundos la potencia sube de 2.35 W a cerca de 6.5 W por nodo mientras cargan la base GTDB desde el NFS, y después vuelve a niveles cercanos al reposo durante la clasificación. Esto muestra que el clúster no es ideal para tareas con alto volumen de acceso a datos, porque todos los nodos dependen de la misma red y del almacenamiento del nodo maestro.

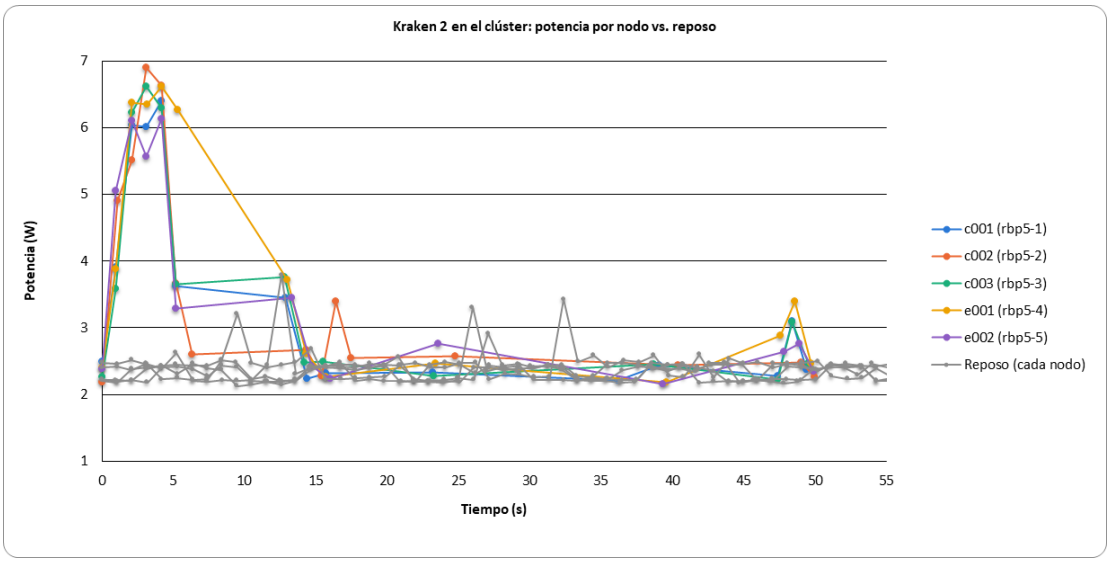


Figura 3.7: Kraken 2 en el clúster: potencia activa de cada nodo durante la ejecución distribuida y línea base de reposo previo (la curva gris es el reposo de 5 minutos medido justo antes de cada corrida).

Para el caso del computador operando con WSL2 (Figura 3.8), la ejecución alterna entre fases de alta demanda (30 a 40 W) durante el cómputo de cada muestra biológica e intervalos de pronunciado descenso hacia la línea base (13.8 W) en las transiciones secuenciales entre muestras.

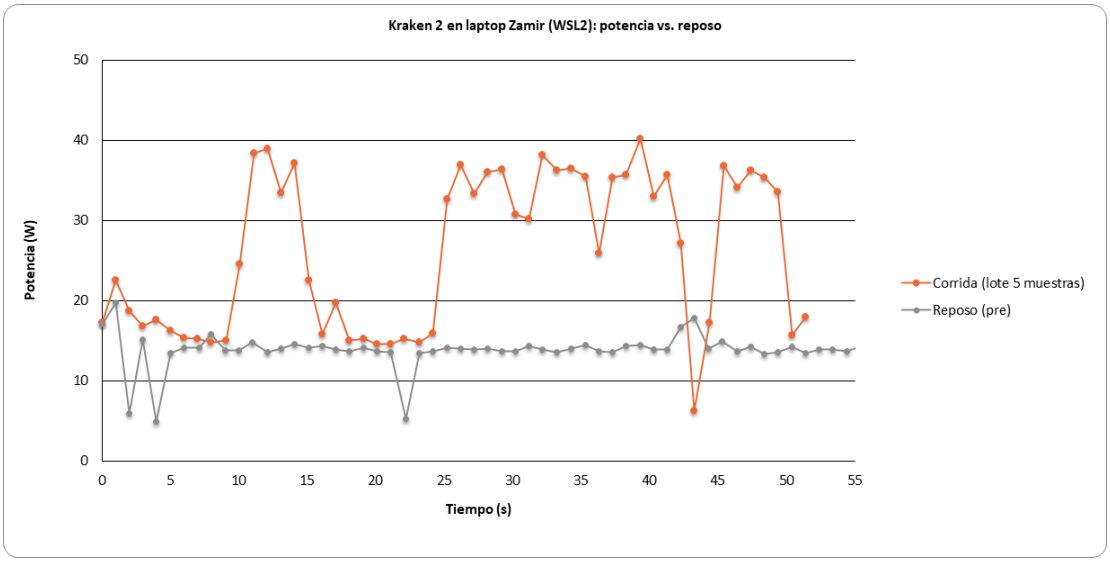


Figura 3.8: Kraken 2 en la laptop con WSL2: potencia durante el procesamiento secuencial de las 5 muestras y reposo previo (gris).

En el caso del computador operando de forma nativa en x86 (Figura 3.9), la máquina opera de manera ininterrumpida sin valles marcados, sosteniendo un consumo de entre 32 y 47 W sobre su base de 10.9 W, lo cual evidencia la utilización continua de los núcleos durante los rápidos 22.0 segundos de procesamiento.

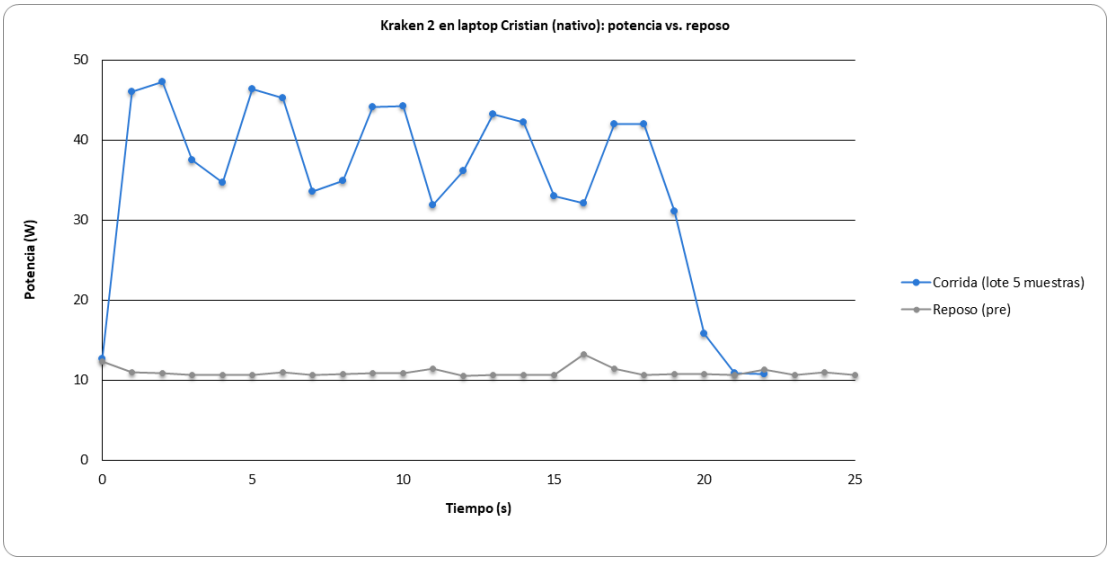


Figura 3.9: Kraken 2 en la laptop con Ubuntu nativo: potencia durante el procesamiento de las 5 muestras y reposo previo (gris).

Al evaluar la potencia neta de cómputo (Figura 3.10), la estación nativa demanda una fuerte tasa instantánea de energía neta (hasta 36 W adicionales a su reposo), lo cual es compensado por su rápida finalización. En contraste, el clúster únicamente disipa potencia normalizada de forma apreciable durante la carga en memoria de la base taxonómica y posteriormente opera con un consumo neto ínfimo, lo que consolida su ahorro de energía limpia a pesar de tomar más tiempo.

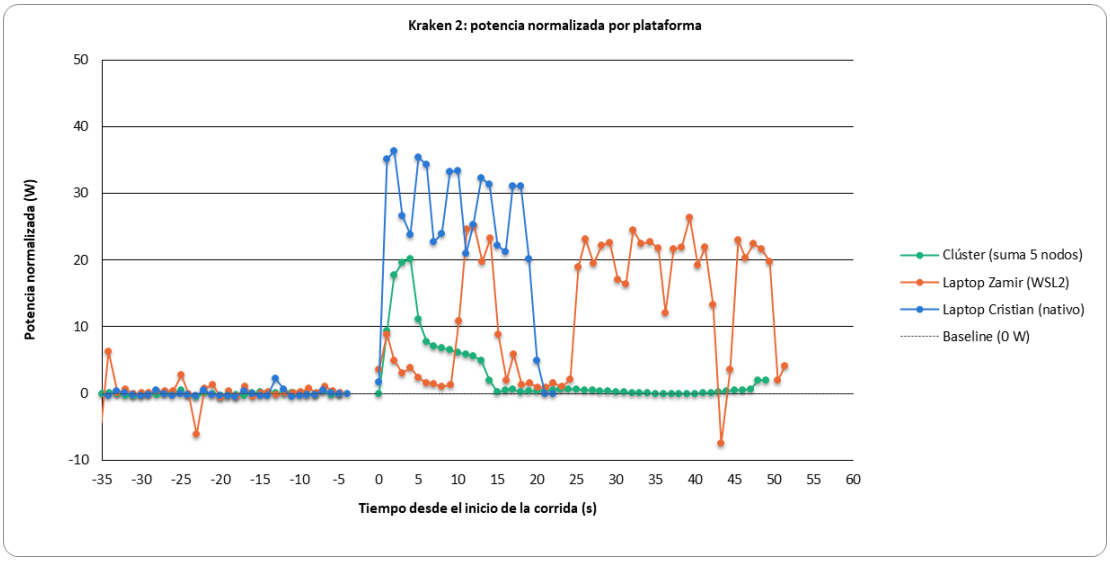


Figura 3.10: Kraken 2: potencia normalizada ($P_{\text{activa}} - \bar{P}_{\text{idle}}$) comparativa entre las tres plataformas evaluadas.

3.4.3. Análisis de desempeño y perfiles de potencia: LotuS3 Suite

La Tabla 3.6 sintetiza las métricas para las 25,611 lecturas PacBio HiFi distribuidas en 5 muestras.

Tabla 3.6: Métricas de desempeño computacional y energético: LotuS3 Suite (5 muestras, 25,611 lecturas PacBio HiFi, protocolo 5 min de reposo basal).

Métrica de Evaluación	**x86 (Ubuntu nativo)**	**x86 (WSL2)**	**Clúster RBPi5**
Tiempo de ejecución (ventana medida, s)	1,899.24 (31.65 min)	1,775.85 (29.60 min)	1,972.46 (32.87 min, paralelo 5 nodos)
Potencia en reposo (P_{idle} 5 min) (W)	2.83	15.20	2.36 (por nodo)
Potencia media en carga (W)	32.48	35.76	6.22 (por nodo)
Potencia neta de cómputo (W)	29.65	20.57	3.86 (por nodo)
Energía total consumida (Wh)*	17.1442	17.6435	12.0311
Energía neta consumida (Wh)*	15.6519	10.1478	7.3270

* Energía acumulada durante la corrida completa (no promedio). En el clúster RBPi5 representa el consumo consolidado de los cinco nodos de cómputo.

En LotuS3 el clúster completó el lote en 32.87 min (1,972.46 s), frente a 31.65 min en la laptop con Ubuntu nativo y 29.60 min en WSL2. El tiempo de cómputo puede explicarse

porque cada lectura cubre el gen 16S completo y LotuS3 formó cerca de 2,150 OTUs por muestra, cada una de las cuales se alineó con `vsearch` contra la base SILVA 138.1. Esa etapa de asignación taxonómica ocupó la mayor parte de la corrida en las tres plataformas. El índice de `vsearch` de la base (unos 3 GB) se construyó una sola vez antes de las mediciones, de modo que el tiempo medido incluye el control de calidad, el agrupamiento en OTUs, el filtrado de quimeras y la asignación taxonómica, pero no la construcción del índice.

En el clúster cada muestra se procesó en un nodo distinto con 4 hilos, pero los nodos terminaron en tiempos muy distintos: entre 14.3 min (`c002` en `rbp5-2`) y 32.6 min (`e002` en `rbp5-5`). Las cinco muestras tienen un tamaño parecido y en la ejecución secuencial cada una tardó entre 27 y 30 min, así que la diferencia no se debió a las muestras, ni tampoco a la temperatura, porque ningún nodo redujo su frecuencia. El desempeño tambien se vio afectado por el sistema de archivo compartido, el cual puede ser accedido por diferentes usuarios del cluster (un caso normal en entornos de computo compartidos). Los nodos más lentos fueron también los de menor potencia durante la etapa de `vsearch` (5.8 W frente a 7.6 W del más rápido), lo que indica que su procesador pasó parte del tiempo esperando datos. Aun así, el clúster consumió menos energía que las dos laptops: 12.03 Wh en total frente a 17.14 Wh de la laptop nativa y 17.64 Wh de WSL2, y 7.33 Wh netos frente a 15.65 y 10.15 Wh. La energía neta de la laptop nativa resulta mayor que la de WSL2 aunque su energía total es casi igual, porque su reposo medido fue mucho menor (2.83 frente a 15.20 W).

La Figura 3.11 muestra el consumo energético en el tiempo de ejecución. Esta muestra como los cinco nodos parten del reposo de unos 2.4 W y en el primer minuto suben a cerca de 6.5 W, durante el filtrado de calidad y el agrupamiento en OTUs. En el minuto 2 o 3 bajan por un momento a 3–4 W, mientras `vsearch` carga la base de referencia desde el almacenamiento compartido, y después se mantienen entre 6 y 7.6 W durante la asignación taxonómica. Cada nodo vuelve al reposo cuando termina su muestra, por eso las curvas tienen longitudes distintas: `rbp5-2` terminó hacia el minuto 14 y `rbp5-5` hacia el minuto 33. El nodo más rápido fue también el de mayor potencia (cerca de 7.6 W), lo que concuerda con la espera por datos descrita antes.

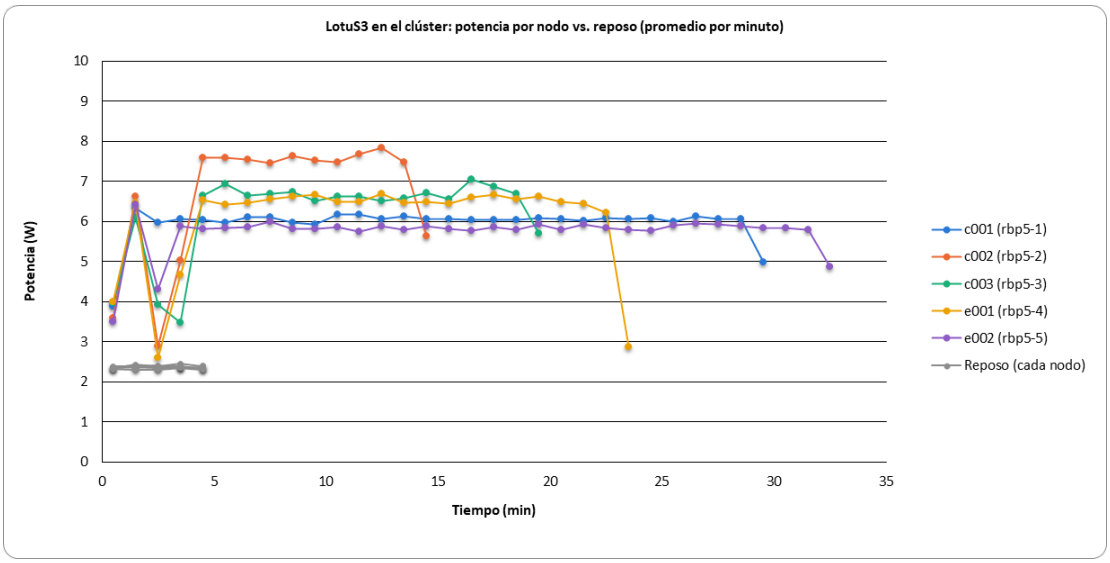


Figura 3.11: LotuS3 en el clúster: promedio por minuto de la potencia de cada nodo y del reposo previo (gris).

La Figura 3.12, muestra el desempeño del computador operando con WSL2. En este caso la potencia se mantiene casi constante en unos 36 W durante los 29.6 min de la corrida, muy por encima del reposo de 15.2 W. Las únicas variaciones son bajadas a unos 32 W cada seis minutos, que coinciden con el paso de una muestra a la siguiente: las cinco muestras tardaron casi lo mismo, entre 346 y 362 s cada una.

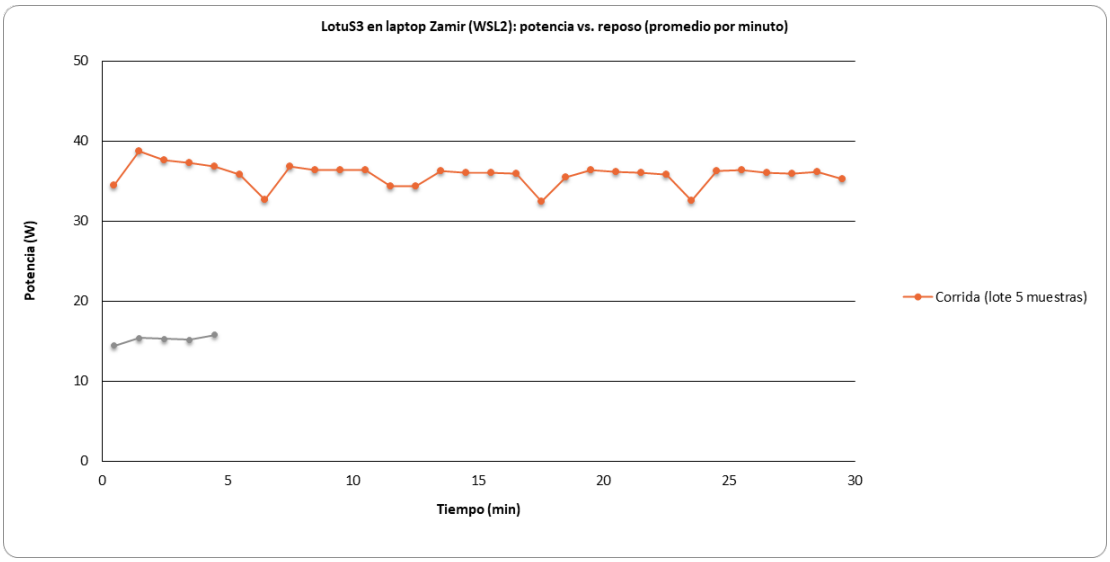


Figura 3.12: LotuS3 en la laptop con WSL2: promedio por minuto de la potencia durante el procesamiento de las 5 muestras y del reposo previo (gris).

La Figura 3.13 muestra el caso del computador operando con Linux se ve el mismo patrón repetido cinco veces, una por muestra: la potencia sube a unos 40–42 W durante los primeros minutos de cada muestra y luego baja a unos 21 W hasta que termina, cada muestra en unos 6.3 min. El reposo previo de esta sesión fue de 2.83 W, por lo que toda la corrida queda muy por encima de la línea base.

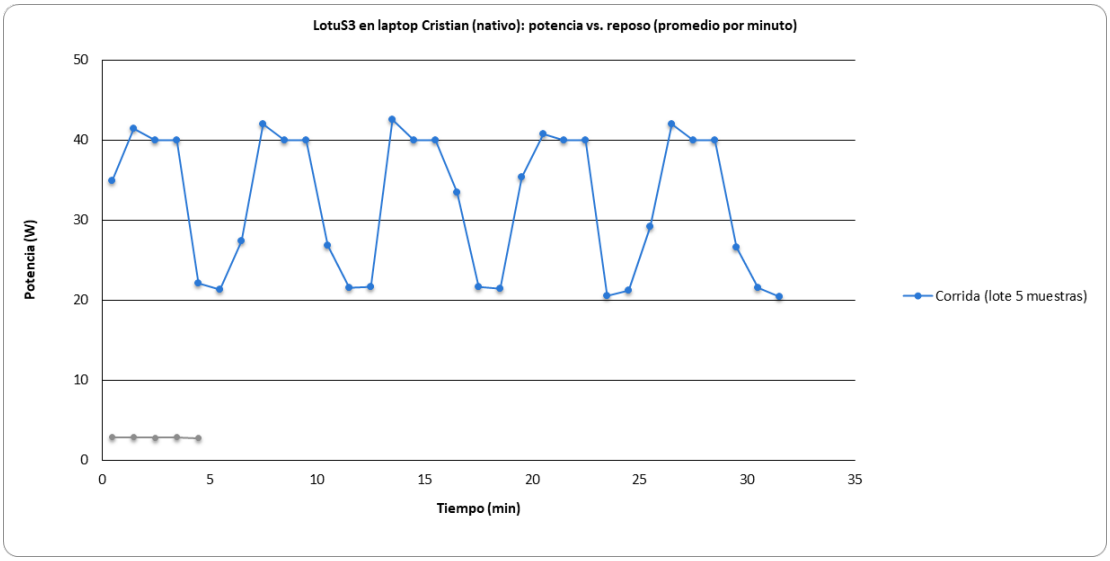


Figura 3.13: LotuS3 en la laptop con Ubuntu nativo: promedio por minuto de la potencia durante el procesamiento de las 5 muestras y del reposo previo (gris).

La Figura 3.14, compara el desempeño de los tres sistemas para el análisis de Lotus3. En la potencia normalizada, WSL2 se mantiene unos 21 W por encima de su reposo durante toda la corrida y la laptop nativa alterna entre unos 37 y 19 W netos según la etapa de cada muestra. La suma de los cinco nodos del clúster llega a unos 21 W netos mientras todos trabajan, un valor parecido al de WSL2, y luego baja en escalones (16, 11, 7 y 3 W) a medida que cada nodo termina su muestra. Por eso, aunque el clúster tardó lo mismo que las laptops, su energía neta fue menor: en la segunda mitad de la corrida solo algunos nodos seguían trabajando.

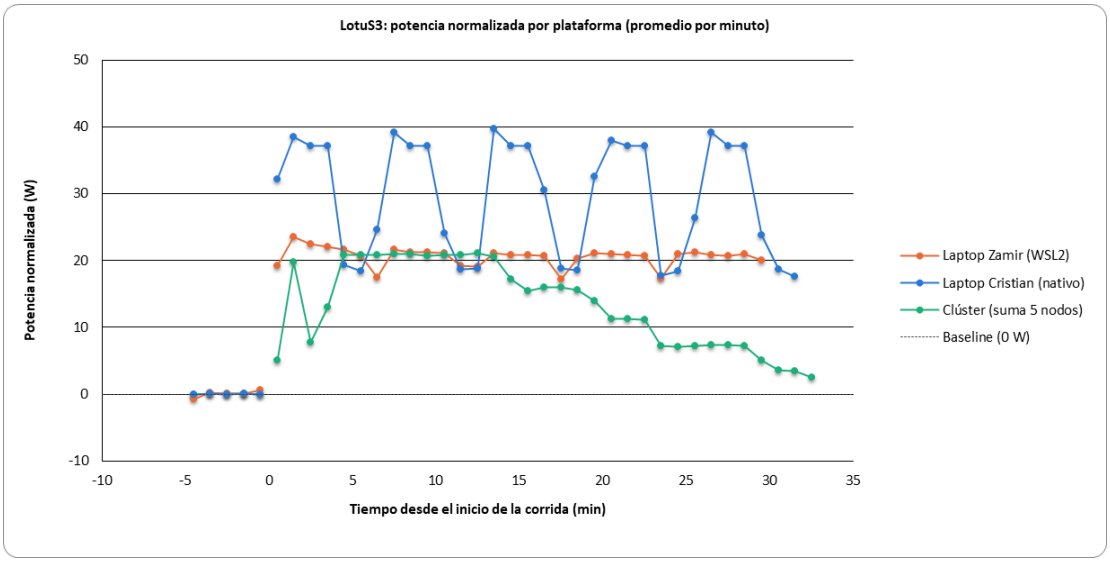


Figura 3.14: LotuS3: promedio por minuto de la potencia normalizada ($P_{\text{activa}} - \bar{P}_{\text{idle}}$) comparativa entre las tres plataformas evaluadas.

3.4.4. Análisis de desempeño y perfiles de potencia: QIIME 2

La Tabla 3.7 abarca los resultados del flujo de trabajo QIIME 2, el pipeline de mayor intensidad y duración computacional del estudio, evaluado sobre 2,690,077 lecturas.

Tabla 3.7: Métricas de desempeño computacional y energético: QIIME 2 (5 muestras, 2,690,077 lecturas, protocolo 5 min de reposo basal).

Métrica de Evaluación	**x86 (Ubuntu nativo)**	**x86 (WSL2)**	**Clúster RBPi5**
Tiempo de ejecución (ventana medida, s)	4,323.72 (72.06 min)	4,799.44 (79.99 min)	1,678.94 (27.98 min)
Uso medio de CPU (%)	96.50	92.00	94.00 (por nodo)
RAM máxima por proceso (GB)	2.40	3.20	2.10 (por nodo)
Potencia en reposo (P_{idle} 5 min) (W)	10.66	12.95	2.42 (por nodo)
Potencia media en carga (W)	21.23	25.68	3.64 (por nodo)
Potencia neta de cómputo (W)	10.57	12.73	1.22 (por nodo)
Energía total consumida (Wh)*	25.4939	34.2407	8.2884
Energía neta consumida (Wh)*	12.6965	16.9772	2.7779

* Energía acumulada durante la corrida completa (no promedio). En el clúster RBPi5 representa el consumo consolidado de los cinco nodos de cómputo.

En QIIME 2, el clúster completó el lote en 27.98 minutos, frente a los 72.06 minutos de x86 nativo y los 79.99 minutos de WSL2, registrando una aceleración de 2.58×.

Cabe resaltar que, para garantizar esta ejecución de forma estable en el hardware ARM de bajo costo, se adoptó una restricción técnica específica: la clasificación taxonómica se evaluó sobre un modelo representativo de 62,000 secuencias de referencia SILVA acotadas a las regiones V1–V2, en lugar de utilizar la base de datos completa no podada (mayor a

500,000 secuencias). Esta decisión limitó el pico de memoria RAM a 2.10 GB por nodo, evitando el agotamiento absoluto de los 16 GB físicos al operar hilos de procesamiento en paralelo.

En el caso de QIIME2, debido a que toma hasta 80 minutos y la telemetría se capturó a 1.0 Hz (generando cerca de 4,800 puntos por curva), las series de potencia a continuación se presentan promediadas en ventanas de un minuto (60 segundos) para maximizar la legibilidad visual. Los cálculos energéticos cuantitativos se mantuvieron, por rigor, basados en la integración numérica completa original a 1.0 Hz.

La Figura 3.15 muestra el desempeño energético del clúster en cada instante de tiempo. Esta indica que los cinco nodos operan con una consistencia notable, manteniendo una media de entre 3.4 y 3.9 W a lo largo de las fases críticas de inferencia. Al culminar las etapas algorítmicas más pesadas alrededor del minuto 26, el consumo desciende en conjunto hasta un nivel de entre 2.5 y 2.9 W, rango que coincide cercanamente con la línea base original antes de dar por terminado el flujo.

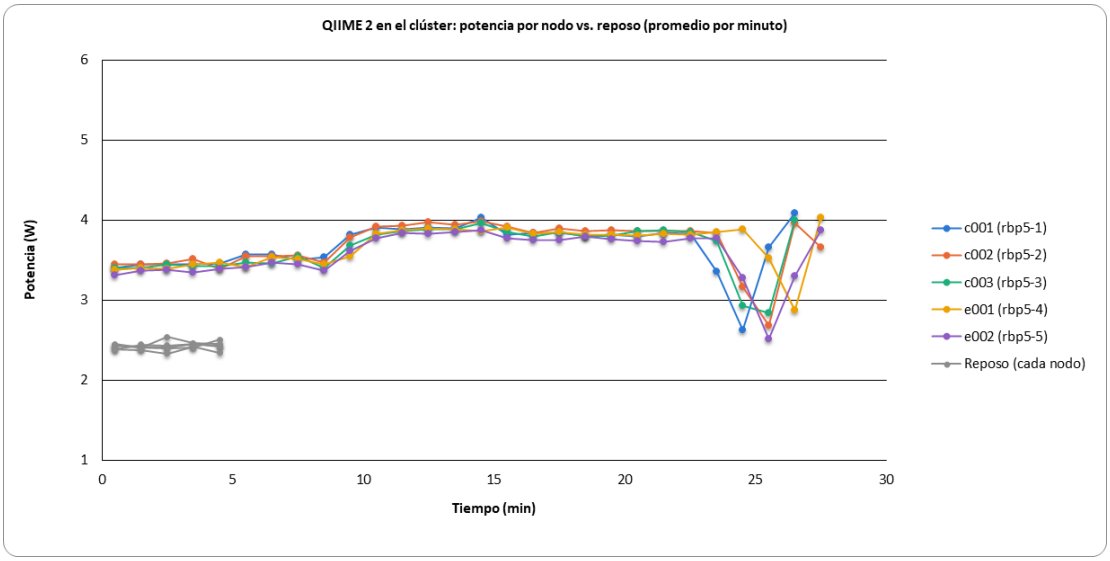


Figura 3.15: QIIME 2 en el clúster: promedio por minuto de la potencia de cada nodo y de la línea base de reposo (gris).

La Figura 3.16 muestra la ejecución bajo WSL2, donde el sistema sostiene el flujo más prolongado y agotador térmicamente, fluctuando en una banda ancha entre 20 y 31 W durante cerca de 80 minutos de ejecución continua.

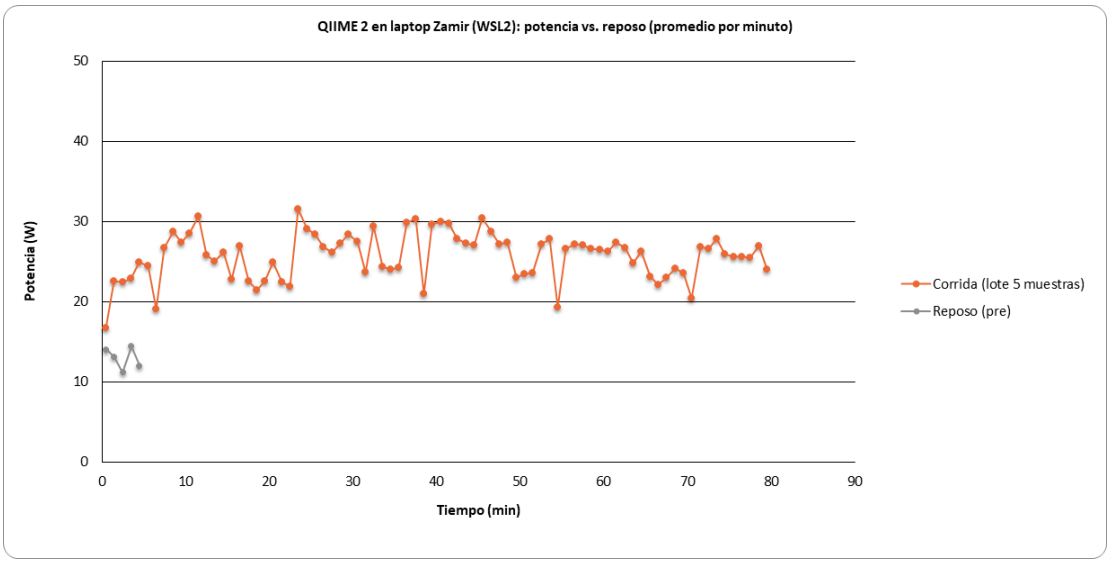


Figura 3.16: QIIME 2 en la laptop con WSL2: promedio por minuto de la potencia y de la línea base de reposo (gris).

La Figura 3.17 muestra la ejecución en Ubuntu nativo, que exhibe una uniformidad en torno a 21 W. Se logran apreciar incrementos rítmicos cada 14 minutos aproximadamente, los cuales coinciden con la culminación de la etapa de modelado secuencial por muestra y el procesamiento del siguiente bloque en el archivo multiplexado.

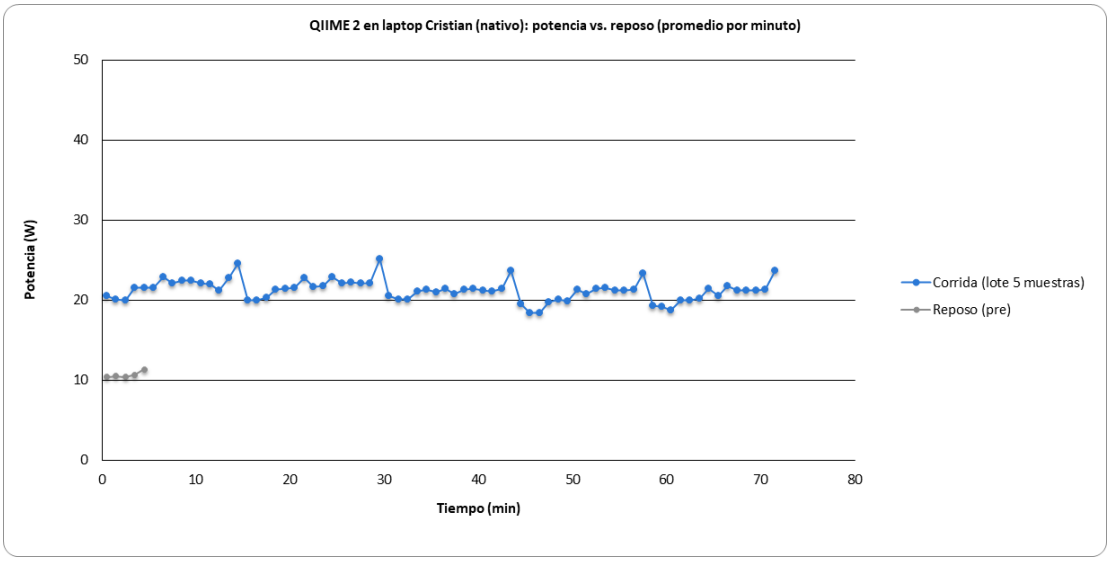


Figura 3.17: QIIME 2 en la laptop con Ubuntu nativo: promedio por minuto de la potencia y de la línea base de reposo (gris).

3.4.4.1. Validez y consistencia del reposo prolongado.

En un experimento con duraciones superiores a una hora, asumir que el nivel de potencia basal permanece constante basado en un reposo de 5 minutos al inicio del ensayo requiere sustentación. Empíricamente, la potencia en reposo de un mismo equipo cambió sutilmente entre sesiones (alrededor de un 5 % en los nodos del clúster, y en un rango ligeramente mayor en la laptop x86 según la actividad de procesos en segundo plano). Por ello, cada corrida se normalizó restando el reposo capturado en su misma sesión. Adicionalmente, un análisis de sensibilidad evidenció que, aun si existiera una deriva térmica residual de ±1.0 W en la computadora portátil a lo largo de los 80 minutos de prueba, los mayores valores de eficiencia neta calculados para el clúster se mantendrían favorables dentro de dicho margen, respaldando la consistencia de la línea base utilizada.

La Figura 3.18 muestra la potencia normalizada del flujo completo y sintetiza de manera observable el impacto operativo del escalamiento ARM64: en conjunto, la potencia algorítmica activa de los cinco nodos físicos sumados se mantuvo contenida entre 5 y 7 W netos, frente a una demanda continua de entre 10 y 17 W netos en la estación x86 de referencia. Como resultado directo de combinar la aceleración temporal por distribución de carga con este bajo perfil de potencia dinámica, el clúster redujo el consumo eléctrico neto final de 12.70 Wh en la laptop nativa a tan solo 2.78 Wh, posicionando una reducción del costo energético superior al 78 % en el modelado biológico más riguroso estudiado.

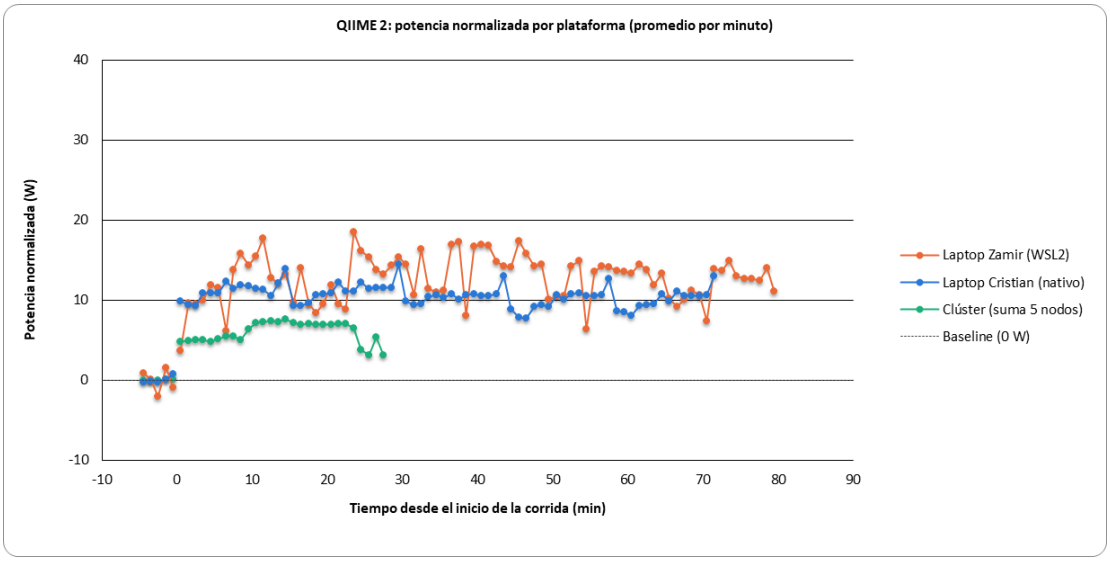


Figura 3.18: QIIME 2: promedio por minuto de la potencia normalizada ($P_{\text{activa}} - \bar{P}_{\text{idle}}$) comparativa.

3.4.5. Eficiencia comparativa global y balance de Green HPC

La integración de las mediciones de los tres pipelines evaluados se consolida a través de la métrica de productividad Reads/Wh en la Tabla 3.8 y se proyecta en la Figura 3.19.

Tabla 3.8: Eficiencia energética comparativa en lecturas biológicas procesadas por vatio-hora (*Reads/Wh*) bajo protocolo unificado de 5 min de reposo.

Pipeline Evaluado	**x86 nativo** (η_{bruta})	**x86 WSL2** (η_{bruta})	**Clúster RBPi5** (η_{bruta})	**Ganancia (RPi5 / Nativo)**
Kraken 2 + Bracken	11,237,105	6,553,358	12,125,061	1.08× **Bruta** (3.81× **Neta**)
LotuS3 Suite	1,494	1,452	2,129	1.42× **Bruta** (2.14× **Neta**)
QIIME 2	105,518	78,564	324,561	3.08× **Bruta** (4.57× **Neta**)

Nota: Las columnas 2 a 4 reportan la productividad energética bruta ($\eta_{\text{bruta}} = N_{\text{reads}}/E_{\text{total}}$). La columna de Ganancia resume la razón calculada frente a x86 nativo tanto en términos brutos ($G_{\text{bruta}} = \eta_{\text{bruta, clúster}}/\eta_{\text{bruta, x86}}$) como netos ($G_{\text{neta}} = E_{\text{neta, x86}}/E_{\text{neta, clúster}}$).

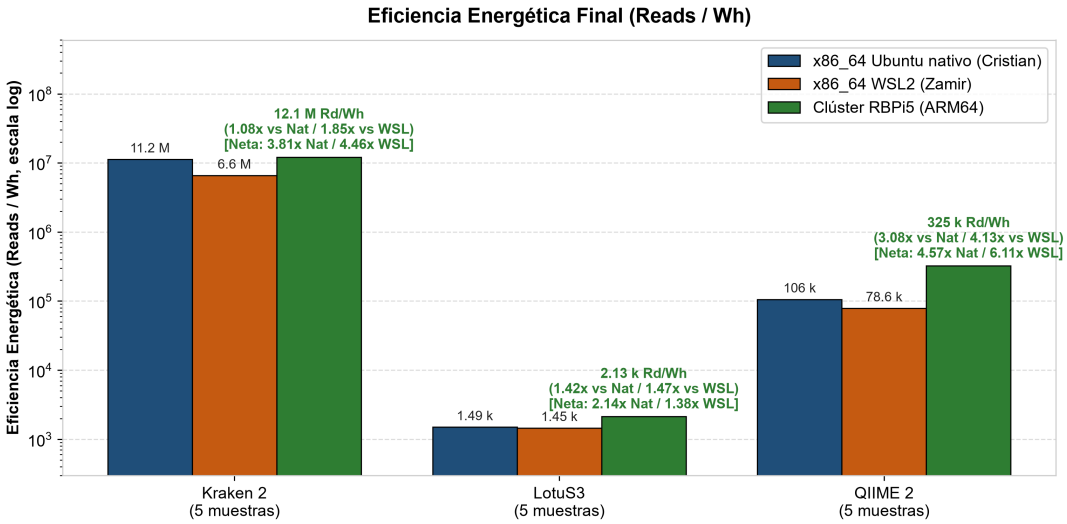


Figura 3.19: Eficiencia energética final ($\eta = N_{\text{reads}}/E$) expresada en lecturas procesadas por vatio-hora (*Reads/Wh*) en escala logarítmica.

La Figura 3.19 muestra la métrica de productividad energética, el clúster Raspberry Pi 5 presentó mayores valores de eficiencia en comparación con la estación x86 de referencia. Las relaciones de eficiencia neta calculadas reportaron factores observados de 3.81×, 2.14× y 4.57× para Kraken 2, LotuS3 y QIIME 2, respectivamente, bajo los instrumentos y el protocolo de telemetría empleados. LotuS3 es el pipeline con menor ventaja, porque el clúster no redujo el tiempo y cada nodo trabajó con sus cuatro núcleos ocupados durante la asignación taxonómica.

Como consideración metodológica final del capítulo, es indispensable reiterar la asimetría de instrumentación existente entre las plataformas: el clúster ARM64 fue monitoreado

a nivel de placa mediante el PMIC Renesas DA9091 de cada Raspberry Pi 5, registrando la potencia de los rieles que alimentan el procesador, la memoria y la circuitería del nodo. En contraste, la interfaz Intel RAPL utilizada en la estación x86 registra el dominio `package-0` (núcleos de CPU y controlador de memoria integrado), sin abarcar la totalidad de las pérdidas de la placa base de la computadora portátil. Dado que los dominios físicos medidos no son idénticos, los factores de eficiencia neta reportados (3.81×, 2.14× y 4.57×) deben interpretarse estrictamente como factores observados bajo los instrumentos y el protocolo de medición utilizados, y no como una equivalencia energética universal entre arquitecturas a nivel de sistema completo.

Capítulo 4

Conclusiones

A partir del diseño metodológico desarrollado, las implementaciones de ingeniería de software ejecutadas y la evidencia experimental recopilada a lo largo del proyecto, se establecen las siguientes conclusiones formales alineadas con los objetivos de la investigación:

4.1. Conclusiones generales

1. **Diseño de arquitectura y caracterización de línea base:** Se diseñó, construyó y validó exitosamente un clúster basado en microcomputadores de placa única Raspberry Pi 5 bajo arquitectura ARM64, orquestado mediante Slurm Workload Manager y enlazado sobre almacenamiento compartido NFS y red Gigabit Ethernet. La caracterización eléctrica en estado de reposo reportó una línea base contenida de 2.38 ± 0.03 W por nodo (11.90 W para los cinco nodos de cómputo utilizados), un consumo bajo que permite mantener el clúster disponible de forma permanente. Este valor corresponde a la placa completa medida por el PMIC y no es directamente equivalente al consumo del encapsulado que reporta RAPL en x86.

2. **Portabilidad y ejecución nativa en ARM64:** Se demostró la viabilidad técnica y operativa de portar y ejecutar de forma nativa los tres pipelines bioinformáticos evaluados (Kraken 2 + Bracken, LotuS3 Suite y QIIME 2) sobre la microarquitectura ARM64, prescindiendo de capas de virtualización o motores de contenedores (como Docker). La suite de herramientas se desplegó exitosamente en espacio de usuario mediante la integración de paquetes de Bioconda para la arquitectura `aarch64`, la compilación nativa de dependencias críticas como `usearch12` a partir de su código fuente y la configuración de QIIME 2 en entornos Python aislados. Asimismo, se garantizó la reproducibilidad

experimental al homologar estrictamente las versiones de software en todas las plataformas (Kraken 2.1.3, Bracken 3.0.1, LotuS 3.03 y QIIME 2 q2cli 2022.8), solventando la falta de compatibilidad del alineador LAMBDA3 en ARM64 mediante el uso estandarizado de VSEARCH. Este escenario demuestra la madurez del ecosistema de software y la viabilidad real de utilizar microarquitecturas ARM en ambientes bioinformáticos con acceso directo al hardware.

3. **Preservación de la integridad y reproducibilidad biológica:** La validación taxonómica frente a los conjuntos de datos de referencia del benchmark LEMMI16s y sus verdades de terreno confirmó que las ejecuciones en procesadores ARM Cortex-A76 preservan la consistencia de las asignaciones en las herramientas evaluadas. En todas las ejecuciones evaluadas, los resultados presentaron concordancia entre las plataformas bajo las configuraciones y versiones utilizadas, sin evidenciar divergencias biológicas atribuibles al cambio de arquitectura. Esto indica que el cambio de arquitectura (de x86 a ARM) no altera la precisión de los métodos de clasificación taxonómica evaluados.

4. **Desempeño computacional y eficiencia energética en Reads/Wh:** El clúster Raspberry Pi 5 presentó mayor eficiencia energética calculada en los tres pipelines. Además, fue posible observar que, al asignar cada muestra a nodos independientes, el clúster redujo el tiempo de procesamiento, superando o igualando los tiempos de ejecución de las plataformas x86 de referencia. En los escenarios con mayor tiempo de cómputo se observó que el tiempo del clúster quedó limitado por la lectura de las bases de referencia desde el almacenamiento compartido. Esto indica que el clúster es más adecuado para cargas de cómputo intensivo, que se reparten bien entre nodos, que para aplicaciones con un alto volumen de acceso a datos compartidos, donde la red y el almacenamiento del nodo maestro se convierten en el cuello de botella.

5. **Respuesta a la pregunta central de investigación:** Los resultados experimentales demuestran la viabilidad técnica y computacional del uso de un clúster basado en microprocesadores de bajo consumo ARM64 (Raspberry Pi 5) para el análisis bioinformático de amplicones 16S. Bajo el protocolo experimental adoptado, la distribución de muestras entre los nodos permitió acelerar sustancialmente los flujos con mayor demanda algorítmica y superó la eficiencia energética de las plataformas x86 de referencia, preservando con exactitud la concordancia taxonómica. La evaluación experimental confirmó que la reducción en los tiempos de cómputo responde directamente al esquema distribuido (5.00× de aceleración en QIIME 2 y 4.35× en LotuS3). Por su parte, la ventaja en eficiencia energética se mantuvo incluso bajo ejecución secuencial: gracias a la reducida potencia activa de cada nodo (entre 2.9 y 6.2 W en carga, frente a los

21 a 36 W de las estaciones x86), el clúster acumuló menor energía total y neta que las laptops en los tres pipelines evaluados. Estos factores son resultados observados con el protocolo de medición adoptado y en estas cargas bioinformáticas; atribuirlos a la arquitectura ARM64 en general requeriría modelos de energía específicos.

4.2. Limitaciones del estudio

Los resultados deben interpretarse dentro de las siguientes limitaciones:

- **Comparación energética no homogénea:** en el clúster la energía se midió con el PMIC a nivel de placa completa, mientras que en x86 RAPL registra el dominio del encapsulado (`package-0`). Como los dominios físicos no son equivalentes, los factores de 3.81×, 2.14× y 4.57× en energía neta deben entenderse como resultados observados con este protocolo y no como equivalencias universales entre arquitecturas.
- **Base de referencia reducida en QIIME 2:** por la memoria disponible, SILVA se redujo a unas 62 000 secuencias de las regiones V1–V2. No se demostró la ejecución con la base completa, que se propone como trabajo futuro.
- **Datos sintéticos:** se usaron conjuntos generados con LEMMI16s como *Ground Truth*. Esto permite validar la reproducibilidad y la fidelidad taxonómica, pero la evaluación con datos biológicos reales fortalecería el trabajo.
- **Almacenamiento compartido:** en Kraken 2 y LotuS3 el tiempo del clúster quedó limitado por la lectura simultánea de las bases de referencia desde NFS sobre la red Gigabit.

4.3. Impacto científico y aplicaciones prácticas

Más allá de los resultados experimentales, este proyecto establece directrices de ingeniería concretas orientadas a democratizar la bioinformática en contextos con limitaciones presupuestales:

- **Accesibilidad económica para instituciones educativas:** Un clúster con cinco nodos de cómputo homogéneos Raspberry Pi 5 (20 núcleos y 80 GB de RAM agregada) y un nodo maestro Raspberry Pi 4 puede implementarse con una inversión sustancialmente menor que la requerida para un servidor de cómputo tradicional. Esta arquitectura ofrece una alternativa de bajo costo para grupos de investigación o instituciones educativas interesadas en implementar infraestructura local de procesamiento para amplicones

16S, disminuyendo la dependencia inicial de recursos en la nube para cargas moderadas.

- **Potencial para bioinformática de campo y sistemas fuera de la red (*Off-Grid*):** Durante las fases de cómputo activo, la potencia agregada de los cinco nodos de cómputo se mantuvo entre 14 y 18 W en Kraken 2 y QIIME 2, y cerca de 31 W en LotuS3, que mantuvo los cuatro núcleos de cada nodo ocupados. Este perfil de consumo sugiere que sería técnicamente interesante evaluar en investigaciones posteriores el acoplamiento del clúster a sistemas fotovoltaicos portátiles (por ejemplo, paneles de 100 W con acumuladores de respaldo) para escenarios de trabajo de campo o estaciones remotas sin acceso confiable a la red eléctrica comercial, si bien la validación práctica de dicha autonomía requerirá ensayos experimentales específicos.

4.4. Trabajo futuro

Como líneas de investigación y desarrollo futuro derivadas de las limitaciones y hallazgos del presente proyecto, se propone:

1. **Escalamiento físico y enlaces de interconexión de alta velocidad:** Evaluar computacionalmente la implementación completa de los 10 nodos físicos creados para el clúster, integrando adaptadores Ethernet 2.5GbE o almacenamiento NVMe acoplado al puerto PCIe de la Raspberry Pi 5. Puesto que los resultados de Kraken 2 y LotuS3 evidenciaron que la red Gigabit (1000BASE-T) y el almacenamiento SD limitan los tiempos iniciales de transferencia masiva de datos, esta mejora permitirá mitigar el cuello de botella de I/O y validar la escalabilidad frente a cargas pesadas de metagenómica *shotgun*.

2. **Portabilidad de algoritmos alternativos de inferencia:** Explorar la portabilidad y compilación nativa del algoritmo DADA2 en el entorno R sobre la arquitectura ARM64. Debido a que esta investigación se enfocó en métodos de agrupamiento (*clustering*) y en Deblur por la complejidad de compilar dependencias biomatemáticas de R en aarch64, lograr la ejecución de DADA2 completaría el espectro analítico, permitiendo contrastar directamente la demanda energética de sus Variantes Exactas de Secuencia (ASVs) frente a los clasificadores evaluados.

3. **Monitoreo autónomo e integración energética solar:** Implementar un sistema de telemetría acoplado a un controlador de carga solar MPPT para la operación del clúster fuera de la red eléctrica (*off-grid*). Ya que el presente trabajo demostró empíricamente la ventaja del modelo de *Green HPC* con consumos bajo carga de entre 2.9 y 6.2 W por nodo, el paso consecuente es llevar esta eficiencia a escenarios reales de bioinformática de campo (computación en el borde o *Edge Computing*), garantizando su viabilidad bajo suministro solar autónomo.

4. **Ejecución de clasificadores taxonómicos densos a escala completa (Full-Scale SILVA):** Diseñar e implementar estrategias de particionamiento dinámico de bases de datos, fragmentación de índices de k-mers o memoria virtual de intercambio (*swap*) de ultra alta velocidad sobre unidades NVMe acopladas al bus PCIe de la Raspberry Pi 5. Esto resolvería la principal limitación observada en QIIME 2, permitiendo ejecutar el flujo completo empleando la base de datos taxonómica SILVA 138 en su totalidad (>500,000 secuencias) sin incurrir en desbordamientos de memoria física (*Out-of-Memory*).

Referencias

Abrahamsson, P., Helmer, S., Phaphoom, N., Nicolodi, L., Preda, N., y Miori, L. (2013). Affordable and energy-efficient cloud computing clusters: The bolzano Raspberry Pi cloud cluster experiment. En *Proceedings of the 2013 IEEE 5th international conference on cloud computing technology and science (CloudCom)* (pp. 170–175). Descargado de https://doi.org/10.1109/CloudCom.2013.121

Benavides, A., Isaza, J. P., Niño-García, J. P., Alzate, J. F., y Cabarcas, F. (2018). CLAME: A new alignment-based binning algorithm allows the genomic description of a novel xanthomonadaceae from the colombian andes. *BMC Genomics*, *19*(Suppl 8), 858. Descargado de https://bmcgenomics.biomedcentral.com/articles/10.1186/s12864-018-5191-y

Bolyen, E., Rideout, J. R., Dillon, M. R., Bokulich, N. A., Abnet, C. C., Al-Ghalith, G. A., ... Caporaso, J. G. (2019). Reproducible, interactive, scalable and extensible microbiome data science using QIIME 2. *Nature Biotechnology*, *37*(8), 852–857. Descargado de https://doi.org/10.1038/s41587-019-0209-9

Callahan, B. J., McMurdie, P. J., Rosen, M. J., Han, A. W., Johnson, A. J. A., y Holmes, S. P. (2016). DADA2: High-resolution sample inference from Illumina amplicon data. *Nature Methods*, *13*(7), 581–583. Descargado de https://doi.org/10.1038/nmeth.3869

Clarridge, J. E. (2004). Impact of 16s rRNA gene sequence analysis for identification of bacteria on clinical microbiology and infectious diseases. *Clinical Microbiology Reviews*, *17*(4), 840–862. Descargado de https://journals.asm.org/doi/10.1128/cmr.17.4.840-862.2004

Cock, P. J. A., Fields, C. J., Goto, N., Heuer, M. L., y Rice, P. M. (2010). The sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic Acids Research*, *38*(6), 1767–1771. Descargado de https://doi.org/10.1093/nar/gkp1137

Cox, S. J., Cox, J. T., Boardman, R. P., Johnston, S. J., Scott, M., y O'Brien, N. S. (2014). Iridis-Pi: A low-cost, compact demonstration cluster. *Cluster Computing*, *17*(2), 349–358. Descargado de https://eprints.soton.ac.uk/362918/

Curry, K. D., Wang, Q., Nute, M. G., Tyshaieva, A., Reeves, E., Soriano, S., y Treangen, T. J. (2022). EMU: Species-level microbial community profiling of full-length 16s rRNA Oxford Nanopore sequencing data. *Nature Methods*, *19*(7), 845–853. Descargado de https://doi.org/10.1038/s41592-022-01520-4

Flórez Suárez, E. A. (2016). *Calendarización energéticamente eficiente en milliclusters* (Tesis de Master no publicada). Universidad Industrial de Santander, Escuela de Ingeniería de Sistemas e Informática. (Directores: Carlos Jaime Barrios Hernández y Johnatan E. Pecero Sánchez)

Fukuda, K., Ogawa, S., Taniguchi, H., y Date, M. (2016). Comparative analysis of 16s rRNA gene amplification and sequencing platforms. *Scientific Reports*, *6*, 36128. doi: 10.1038/srep36128

Gómez-Sánchez, G., Berral, J. L., Torrents, D., y Pérez, A. (2023). Challenges and opportunities for RISC-V architectures towards genomics-based workloads. En A. Bienz, M. Weiland, M. Baboulin, y C. Kruse (Eds.), *High performance computing. ISC high performance 2023* (Vol. 13999). Springer. Descargado de https://doi.org/10.1007/978-3-031-40843-4_34

Hildebrand, F., y Özkurt, E. (2024). *LotuS3: Less OTU scripts version 3.* https://github.com/hildebra/LotuS3. GitHub. (Versión 3.03)

Jain, M., Olsen, H. E., Paten, B., y Akeson, M. (2016). The Oxford Nanopore MinION: Delivery of nanopore sequencing to the genomics community. *Genome Biology*, *17*(1), 239. Descargado de https://link.springer.com/article/10.1186/s13059-016-1103-0

Lee, J., Jeong, I., Kim, J.-O., y Kim, K. (2021). Microbial diversity analysis of sediment from yeosu new harbor of south korea using 16s rRNA gene amplicon sequencing. *Microbiology Resource Announcements*, *10*(1), e01229-20. Descargado de https://journals.asm.org/doi/full/10.1128/mra.01229-20

Maqbool, J., Oh, S., y Fox, G. C. (2015). Evaluating ARM HPC clusters for scientific workloads. *Concurrency and Computation: Practice and Experience*. Descargado de https://doi.org/10.1002/cpe.3602

Miller, R. (2017). The Raspberry Pi tackles HPC with 750-unit cluster. *Data Center Frontier*. Descargado de https://www.datacenterfrontier.com/servers/article/11430486/the-raspberry-pi-tackles-hpc-with-750-unit-cluster

Oxford Nanopore Technologies. (2024). *wf-metagenomics: A Nextflow pipeline for metagenomic classification using Kraken2 and Bracken.* GitHub. Descargado de https://github.com/epi2me-labs/wf-metagenomics

Özkurt, E., Fritscher, J., Soranzo, N., Ng, D. Y., Davey, R. P., y Bahram, M. (2022). LotuS2:

An ultrafast and highly accurate tool for amplicon sequencing analysis. *Microbiome*, *10*(1), 176. Descargado de https://doi.org/10.1186/s40168-022-01365-1

Purdue University. (2021). *What is Six Sigma?* (Inf. Téc.). Purdue University. Descargado de https://www.purdue.edu/leansixsigmaonline/blog/what-is-six-sigma/

Pyrhönen, J., y Ikonen, J. (2025). Energy-saving techniques for Raspberry Pi. En *2025 international conference on computer systems and technologies (CompSysTech)*. Descargado de https://www.researchgate.net/publication/395210335_Energy-Saving_Techniques_for_Raspberry_PI

Quince, C., Walker, A. W., Simpson, J. T., Loman, N. J., y Segata, N. (2017). Shotgun metagenomics, from sampling to analysis. *Nature Biotechnology*, *35*(9), 833–844. Descargado de https://www.sci-hub.in/10.1038/nbt.3935

Rajovic, N., Rico, A., Puzovic, N., Adeniyi-Jones, C., y Ramirez, A. (2014). Tibidabo: Making the case for an ARM-based HPC system. *Future Generation Computer Systems*, *36*, 322–334.

Raspberry Pi Ltd. (2023). Raspberry Pi 5 product brief [Manual de software informático]. Descargado de https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf

Robinson, T., Harkin, J., y Shukla, P. (2021). Hardware acceleration of genomics data analysis: Challenges and opportunities. *Bioinformatics*, *37*(13), 1785–1795. Descargado de https://academic.oup.com/bioinformatics/article/37/13/1785/6284128

Scientific Community and HPC Researchers. (2025). *Apple vs. oranges: Evaluating Apple Silicon for HPC*. Descargado de https://arxiv.org/abs/2502.05317

Sehli, S., Idrissi Azami, A., y Dini, N. (2022). Gut microbiome 16s rRNA gene amplicon taxonomic profiling of hospitalized moroccan COVID-19 patients. *Microbiology Resource Announcements*, *11*(7), e00256-22. Descargado de https://journals.asm.org/doi/10.1128/mra.00256-22

Seppey, M., Benavides, A., Berkeley, M. R., Manni, M., y Zdobnov, E. M. (2025). LEMMIv2: Benchmarking framework for metagenomic and 16s amplicon profilers with a catalogue of evaluated tools. *bioRxiv*. Descargado de https://www.biorxiv.org/content/10.1101/2025.03.06.641904v1 (Preprint)

Sterling, T., Becker, D. J., Savarese, D., Dorband, J. E., Ranawake, U. A., y Packer, C. V. (1995). BEOWULF: A parallel workstation for scientific computation. En *Proceedings of the 24th international conference on parallel processing (ICPP)* (Vol. 1, pp. 11–14).

Vanguardia. (2012). *La UIS tiene el computador más potente de Colombia.* Vanguardia, 23 de abril de 2012. Descargado de https://www.vanguardia.com/area-metropolitana/bucaramanga/la-uis-tiene-el-computador-mas-potente

-de-colombia-KFVL153559

Warade, M., Schneider, J.-G., y Lee, K. (2022). Measuring the energy and performance of scientific workflows on low-power clusters. *Electronics*, *11*(11), 1801. Descargado de https://www.mdpi.com/2079-9292/11/11/1801

Wood, D. (2024). Kraken 2 manual [Manual de software informático]. Descargado de https://github.com/DerrickWood/kraken2/wiki/Manual

Wood, D. E., Lu, J., y Langmead, B. (2019). Improved metagenomic analysis with Kraken 2. *Genome Biology*, *20*(1), 257. Descargado de https://doi.org/10.1186/s13059-019-1891-0

Zhang, H., Wang, Y., Li, X., Chen, J., y Liu, M. (2025). Human activities have noticeably reduced microbial diversity in water bodies across the yunnan-guizhou plateau region. *Annals of Microbiology*, *75*(1), 18. Descargado de https://link.springer.com/article/10.1186/s13213-025-01806-w

Repositorio y recursos complementarios

El código fuente desarrollado en este proyecto de grado, los scripts de sumisión y orquestación Slurm, los archivos de configuración de entornos nativos para ARM64, la suite de telemetría energética y los conjuntos de datos procesados se encuentran disponibles de manera pública y abierta para fines de reproducibilidad científica en el siguiente repositorio:

https://github.com/CristianSolanoUIS/lemmi16s-cluster-RbPi5

Adicionalmente, la documentación metodológica, guías de instalación paso a paso de cada pipeline y las instrucciones de despliegue del clúster se detallan en el archivo `README.md` y en las guías técnicas del proyecto.